

Pontificia Universidad Javeriana

Facultad de Ingeniería

Ingeniería de Sistemas

SecureTicket

Solución B2B de reventa de entradas basada en
contratos inteligentes sobre Stellar/Soroban

Plan de Pruebas

Versión del documento: v2.0

Fecha: 24 de mayo de 2026

Autores: Juan Diego Arias Durán

Santiago Andrés Mesa Niño

Andrés Felipe Manosalva Garzón

Juan Camilo Carvajal Rodríguez

Director: Rafael Vicente Páez Méndez, PhD

Bogotá, D.C.

2026

Historial de cambios

Versión	Fecha	Sección modificada	Descripción de cambios	Responsable
1.0	18/05/2026	Inicio del documento	Versión inicial del plan de pruebas de Secure Ticket.	Juan Camilo Carvajal Rodríguez
2.0	19/05/2026	Documento completo	Corrección integral del plan de pruebas: ajuste de formato de tablas, numeración de tablas con leyendas inferiores, actualización de secciones de estrategia, ambiente, criterios, casos de prueba, resultados, riesgos, trazabilidad, conclusiones y anexos; incorporación de resultados actualizados de pruebas automatizadas locales, evidencias persistidas, matriz de trazabilidad y delimitación de pruebas manuales pendientes.	Juan Camilo Carvajal Rodríguez

Índice

Historial de cambios	1
1. Introducción	9
1.1. Propósito del documento	9
1.2. Alcance del sistema	9
1.3. Audiencia objetivo	10
2. Objetivos del plan de pruebas	11
2.1. Objetivo general	12
2.2. Objetivos específicos	12
2.3. Metas de calidad	13
2.4. Criterios generales de éxito	14
3. Ítems de prueba	14
4. Alcance de las pruebas	18
4.1. Componentes incluidos en el alcance	18
4.2. Flujos funcionales incluidos	19
4.3. Niveles de prueba incluidos	20
4.4. Cobertura parcial por diseño	20
4.5. Elementos fuera del alcance	21
4.6. Criterio de delimitación del alcance	22
5. Estrategia de pruebas	22
5.1. Pruebas unitarias de contratos inteligentes	22
5.2. Pruebas unitarias y de API del backend	23
5.3. Pruebas de integración	24

5.4. Pruebas end-to-end del frontend	25
5.5. Prueba manual guiada de reventa en Testnet	25
5.6. Pruebas del escáner QR	26
5.7. Pruebas de seguridad funcional	27
5.8. Pruebas de carga	28
5.9. Pruebas de humo de despliegue	28
5.10. Pruebas de aceptación de usuario	29
5.11. Trazabilidad de pruebas	30
6. Ambiente y datos de prueba	30
6.1. Ambientes de prueba	31
6.2. Configuración técnica del ambiente	31
6.3. Variables de entorno	32
6.4. Datos de prueba	33
6.5. Estados de ticket considerados	34
6.6. Datos para pruebas de carga	35
6.7. Consideraciones de seguridad sobre datos de prueba	35
6.8. Criterios de preparación del ambiente	36
7. Criterios de entrada y salida	37
7.1. Criterios generales de entrada	37
7.2. Criterios generales de salida	38
7.3. Criterios de entrada por tipo de prueba	38
7.4. Criterios de salida por tipo de prueba	39
7.5. Estados de resultado de prueba	40
7.6. Criterio de suspensión y reanudación	41
8. Casos de prueba	41
8.1. Formato de los casos de prueba	42

8.2. Casos de prueba de contratos inteligentes	42
8.3. Casos de prueba backend/API	43
8.4. Casos de prueba de integración	44
8.5. Casos de prueba frontend y end-to-end	45
8.6. Caso de prueba manual de reventa en Testnet	46
8.7. Casos de prueba de carga	47
8.8. Casos de prueba de smoke test de despliegue	47
8.9. Casos de prueba de aceptación de usuario	48
8.10. Casos documentales de trazabilidad	48
8.11. Resumen de cobertura de casos	49
9. Resultados y análisis	50
9.1. Resumen general de resultados	50
9.2. Resultados de pruebas de contratos inteligentes	51
9.3. Resultados de pruebas Backend/API	52
9.4. Resultados de integración e indexador	53
9.5. Resultados de frontend y pruebas end-to-end	54
9.6. Resultados de reventa en Stellar/Soroban Testnet	54
9.7. Resultados del escáner QR	55
9.8. Resultados de pruebas de carga	56
9.9. Resultados del smoke test de despliegue	57
9.10. Resultados de aceptación de usuario	57
9.11. Resultados de trazabilidad	58
9.12. Fallos, bloqueos y limitaciones identificadas	59
9.13. Análisis general	60
9.14. Conclusión de resultados	61
10. Riesgos y mitigaciones	62

10.1. Riesgos identificados	63
10.2. Plan de mitigación	64
10.3. Riesgos materializados durante la ejecución	66
10.4. Plan de contingencia	67
10.5. Riesgo residual	68
10.6. Conclusión de riesgos	68
11. Matriz de trazabilidad	68
11.1. Criterios de trazabilidad	69
11.2. Estados utilizados en la matriz	70
11.3. Ubicación de la matriz completa	70
11.4. Resumen de cobertura por categoría	71
11.5. Análisis de brechas de trazabilidad	71
11.6. Conclusión de trazabilidad	72
12. Conclusiones	73
12.1. Trabajo futuro	74
13. Anexos	75
13.1. Anexo A: Evidencias de pruebas de contratos inteligentes	75
13.2. Anexo B: Evidencias de pruebas backend/API e integración	75
13.3. Anexo C: Evidencias de pruebas frontend y end-to-end	76
13.4. Anexo D: Evidencias de reventa en Stellar/Soroban Testnet	77
13.5. Anexo E: Evidencias de pruebas de carga	77
13.6. Anexo F: Matriz completa de trazabilidad	78
13.7. Anexo G: Evidencias de smoke test de despliegue	80
13.8. Anexo H: Formularios de aceptación de usuario	80
13.9. Anexo I: Limitaciones y decisiones de alcance	81

Índice de tablas

1.	Referencias utilizadas en el plan de pruebas	11
2.	Criterios generales de éxito del plan de pruebas	14
3.	Ítems de prueba definidos para Secure Ticket	17
4.	Flujos funcionales incluidos en el alcance de pruebas	19
5.	Elementos con cobertura parcial por diseño	21
6.	Ambientes definidos para la ejecución de pruebas	31
7.	Configuración técnica del ambiente de pruebas	32
8.	Variables de entorno requeridas para pruebas	33
9.	Datos de prueba funcionales definidos para los escenarios principales	34
10.	Datos de prueba negativos y de integración	34
11.	Estados de ticket considerados durante las pruebas	35
12.	Datos requeridos para pruebas de carga	35
13.	Criterios generales de entrada para la ejecución de pruebas	37
14.	Criterios generales de salida del proceso de validación	38
15.	Criterios de entrada por tipo de prueba	39
16.	Criterios de salida por tipo de prueba	40
17.	Estados utilizados para registrar resultados de prueba	40
18.	Formato utilizado para documentar casos de prueba	42
19.	Casos de prueba de contratos inteligentes	43
20.	Casos de prueba backend/API	44
21.	Casos de prueba de integración	45
22.	Casos de prueba frontend y end-to-end	46
23.	Caso de prueba manual de reventa en Testnet	47
24.	Casos de prueba de carga	47
25.	Casos de prueba de smoke test de despliegue	48

26.	Casos de prueba de aceptación de usuario	48
27.	Casos documentales de trazabilidad	49
28.	Resumen de cobertura esperada por categoría de prueba	49
29.	Resumen general actualizado de resultados de pruebas	51
30.	Resultados de pruebas de contratos inteligentes	52
31.	Resultados actualizados de pruebas Backend/API	53
32.	Resultados actualizados de integración e indexador	53
33.	Resultados actualizados de pruebas frontend/E2E	54
34.	Estado de la prueba manual de reventa en Testnet	55
35.	Resultados actualizados del escáner QR	55
36.	Resultados actualizados de pruebas de carga	56
37.	Estado del smoke test autenticado en staging	57
38.	Estado actualizado de pruebas de aceptación de usuario	58
39.	Resultados actualizados de trazabilidad	59
40.	Bloqueos automatizables resueltos durante la segunda ejecución	60
41.	Pendientes, dependencias externas y limitaciones de alcance restantes	60
42.	Riesgos identificados durante el ciclo de pruebas, parte 1	63
43.	Riesgos identificados durante el ciclo de pruebas, parte 2	64
44.	Riesgos identificados durante el ciclo de pruebas, parte 3	64
45.	Plan de mitigación de riesgos, parte 1	65
46.	Plan de mitigación de riesgos, parte 2	65
47.	Plan de mitigación de riesgos, parte 3	66
48.	Riesgos materializados durante la ejecución, parte 1	66
49.	Riesgos materializados durante la ejecución, parte 2	67
50.	Riesgos residuales	68
51.	Estados utilizados en la matriz de trazabilidad	70
52.	Resumen de cobertura por categoría	71

Índice de figuras

1. Introducción

1.1. Propósito del documento

El propósito de este documento es definir el plan de pruebas de SecureTicket, estableciendo los ítems de prueba, la estrategia de validación, los tipos de prueba aplicables, los criterios de aceptación, los casos de prueba y el mecanismo de registro de los resultados obtenidos durante la verificación y validación del prototipo.

Este plan sirve como guía para evaluar que los componentes principales del sistema cumplen con los requerimientos funcionales y no funcionales definidos para el proyecto. En particular, busca validar los flujos críticos asociados a la autenticidad, trazabilidad, reventa y verificación de tickets mediante una arquitectura híbrida compuesta por frontend web, backend, base de datos PostgreSQL y contratos inteligentes ejecutados sobre Stellar/Soroban Testnet.

El documento también establece una base objetiva para reportar el estado de calidad del software antes de la entrega final del trabajo de grado, diferenciando entre pruebas automatizadas, pruebas manuales guiadas, pruebas de integración, pruebas de carga, pruebas de aceptación y limitaciones propias del alcance académico del prototipo.

1.2. Alcance del sistema

SecureTicket es un prototipo académico orientado a soportar la reventa segura de tickets para eventos mediante el uso de contratos inteligentes y tokens no fungibles sobre la red Stellar/Soroban. El sistema permite representar tickets como activos digitales, gestionar su estado, consultar su trazabilidad, habilitar flujos de reventa y verificar tickets mediante un flujo de escaneo QR.

El alcance de este plan cubre las pruebas sobre los siguientes componentes:

- Contratos inteligentes Soroban asociados a la emisión, reventa, cancelación, invalidación y redención de tickets.
- Backend/API responsable de la lógica de negocio, autenticación, autorización por roles, gestión de tickets, escáner QR, validaciones de estado y comunicación con servicios de persistencia.

- Indexador o mecanismo de sincronización entre eventos on-chain y el estado almacenado off-chain en PostgreSQL.
- Base de datos PostgreSQL utilizada para persistir usuarios, eventos, tickets, órdenes, estados y campos asociados a la integración Web3.
- Frontend web utilizado por compradores, vendedores, administradores y personal de verificación.
- Flujos end-to-end del prototipo, incluyendo compra primaria simulada, inventario, escáner QR y reventa en Stellar/Soroban Testnet.
- Pruebas de seguridad funcional enfocadas en roles, tokens, propiedad de tickets y validación de operaciones no autorizadas.
- Pruebas de carga sobre endpoints REST representativos, sin incluir carga masiva sobre transacciones blockchain.
- Pruebas de aceptación con usuarios o stakeholders en un ambiente académico controlado.

Las pruebas blockchain se realizan exclusivamente sobre Stellar/Soroban Testnet. El despliegue en mainnet, el uso de activos con valor real, pagos reales, integraciones con pasarelas de pago productivas, procesos KYC/AML y operación comercial en producción se encuentran fuera del alcance de este plan de pruebas.

Adicionalmente, algunas funcionalidades se validan de acuerdo con el alcance real del prototipo. La compra primaria se evalúa como un flujo simulado/off-chain cuando así corresponde a la implementación actual. El escáner QR se evalúa en modalidad DB-first, mientras que la redención on-chain se valida a nivel de contrato inteligente. La reventa con Freightier se contempla como prueba manual guiada sobre Stellar/Soroban Testnet, debido a la fragilidad técnica de automatizar extensiones de wallet en pruebas end-to-end.

1.3. Audiencia objetivo

Este documento está dirigido a los siguientes perfiles relacionados con el proyecto SecureTicket:

- Equipo de desarrollo frontend, backend y contratos inteligentes.

- Equipo responsable de pruebas, validación y documentación.
- Director del trabajo de grado.
- Jurados o revisores académicos del proyecto.
- Interesados técnicos que requieran comprender el alcance, la estrategia y los resultados de las pruebas realizadas.

ID	Referencia
R1	Especificación de Requerimientos de Software de SecureTicket (SRS).
R2	Documento de Especificación del Diseño o Arquitectura de Software de SecureTicket (SAD/SDD).
R3	Repositorio oficial del proyecto SecureTicket.
R4	Documentación oficial de Stellar y Soroban para contratos inteligentes y ejecución sobre Testnet.
R5	Documentación de herramientas de prueba utilizadas en el proyecto: Cargo Test, Jest, Supertest, Playwright, k6 y herramientas asociadas al entorno de desarrollo.
R6	ISO/IEC/IEEE 29148:2018, Systems and software engineering – Life cycle processes – Requirements engineering.

Tabla 1: Referencias utilizadas en el plan de pruebas

2. Objetivos del plan de pruebas

El objetivo principal de este plan de pruebas es establecer una estrategia sistemática para verificar y validar el funcionamiento de SecureTicket, considerando los componentes de software que conforman el prototipo y los flujos críticos definidos para el trabajo de grado. Las pruebas buscan comprobar que el sistema cumple con los requerimientos funcionales y no funcionales asociados a la autenticidad, trazabilidad, reventa y verificación de tickets en un ambiente académico controlado.

Dado que SecureTicket integra componentes tradicionales de software con contratos inteligentes sobre Stellar/Soroban Testnet, el plan de pruebas contempla una validación en diferentes niveles: contratos inteligentes, backend/API, frontend web, persistencia, sincronización on-chain/off-chain, pruebas end-to-end, seguridad funcional, carga y aceptación de usuario.

2.1. Objetivo general

Definir y ejecutar una estrategia de pruebas que permita evaluar la calidad funcional, técnica y operativa del prototipo SecureTicket, verificando que sus componentes principales soporten los flujos de compra primaria simulada, reventa, verificación mediante código QR, control de acceso, trazabilidad e interacción con contratos inteligentes sobre Stellar/Soroban Testnet.

2.2. Objetivos específicos

Los objetivos específicos del presente plan de pruebas son:

- Verificar el correcto funcionamiento de los contratos inteligentes Soroban asociados a la emisión, reventa, cancelación, invalidación y redención de tickets.
- Validar que el backend/API implemente correctamente las reglas de negocio relacionadas con autenticación, autorización por roles, gestión de tickets, escáner QR, compra primaria simulada, reventa y restricciones de operación.
- Comprobar que el frontend web permita ejecutar los flujos principales del prototipo desde la perspectiva de los usuarios definidos: comprador, vendedor, administrador y personal de verificación.
- Evaluar la sincronización entre los eventos o estados derivados de la lógica on-chain y la información persistida off-chain en PostgreSQL.
- Validar que el escáner QR permita verificar tickets válidos y rechazar tickets usados, inválidos, inexistentes, vencidos o procesados por usuarios no autorizados.
- Verificar que los mecanismos de seguridad funcional impidan operaciones no autorizadas, incluyendo accesos sin token, tokens inválidos, roles insuficientes, manipulación de solicitudes y operaciones sobre tickets ajenos.
- Evaluar el comportamiento del sistema bajo carga moderada sobre endpoints REST representativos, excluyendo carga masiva sobre transacciones blockchain, Freightier o mainnet.
- Validar la disponibilidad básica del prototipo en el ambiente de despliegue mediante pruebas de humo sobre los servicios principales.

- Obtener una validación académica de aceptación por parte de usuarios o stakeholders mediante tareas representativas del flujo de negocio.
- Registrar los resultados de las pruebas de forma trazable, relacionando cada prueba con los casos de uso, requerimientos o atributos de calidad que busca verificar.

2.3. Metas de calidad

Las metas de calidad que orientan este plan de pruebas son las siguientes:

- **Corrección funcional:** los flujos principales del prototipo deben ejecutarse de acuerdo con las reglas de negocio definidas para compra primaria simulada, reventa, verificación, invalidación, cancelación y redención de tickets, según el alcance implementado.
- **Integridad:** las operaciones sobre tickets deben preservar estados consistentes, evitando que un ticket usado, invalidado, cancelado o no perteneciente al usuario pueda ser operado indebidamente.
- **Trazabilidad:** el sistema debe permitir relacionar el estado de un ticket con su ciclo de vida, incluyendo propietario, versión, estado de venta, eventos relevantes y vínculo con la lógica blockchain cuando aplique.
- **Seguridad funcional:** las acciones críticas deben estar restringidas según rol, token, propiedad del ticket y validación de wallet cuando corresponda.
- **Confiabilidad operativa:** los flujos críticos deben manejar errores esperados de forma controlada, incluyendo códigos QR inválidos, doble escaneo, tickets inexistentes, tokens inválidos y solicitudes manipuladas.
- **Rendimiento básico:** los endpoints REST representativos deben responder de manera estable bajo una carga moderada, dentro de umbrales acordes al alcance académico del prototipo.
- **Usabilidad básica:** los usuarios deben poder completar tareas representativas del sistema, como comprar un ticket, consultar inventario, listar una reventa o escanear un código QR, con una comprensión razonable del estado y resultado de cada operación.

2.4. Criterios generales de éxito

Los criterios generales de éxito permiten establecer las condiciones mínimas bajo las cuales el proceso de pruebas se considera satisfactorio para el alcance académico de SecureTicket. Estos criterios no implican operación productiva ni despliegue en mainnet, sino validación del prototipo en los ambientes definidos para el proyecto.

Categoría	Criterio de éxito	Tipo de validación
Contratos inteligentes	Las pruebas de contratos Soroban deben ejecutarse sin fallos bloqueantes, cubriendo emisión, reventa, cancelación, invalidación, redención y casos negativos relevantes.	Automatizada
Backend/API	Las pruebas unitarias y de API deben validar reglas de negocio, autenticación, autorización, escáner QR, operaciones sobre tickets y manejo de errores esperados.	Automatizada
Frontend/E2E	Los flujos principales del prototipo deben poder ejecutarse desde la interfaz web, incluyendo compra primaria simulada, inventario y escáner QR.	Automatizada / Manual
Reventa en Testnet	La reventa debe validarse mediante una prueba manual guiada sobre Stellar/Soroban Testnet, usando Freightier y verificando cambio de propietario, trazabilidad y actualización del estado del ticket.	Manual guiada
Escáner QR	El escáner debe aceptar tickets válidos y rechazar tickets usados, inválidos, inexistentes, expirados o procesados por usuarios no autorizados.	Automatizada / E2E
Redención on-chain	La redención on-chain debe estar validada a nivel de contrato inteligente, aunque el escáner operativo del prototipo funcione bajo un enfoque DB-first.	Automatizada

Tabla 2: Criterios generales de éxito del plan de pruebas

3. Ítems de prueba

Los ítems de prueba corresponden a los componentes, servicios, flujos, integraciones y artefactos de SecureTicket que serán evaluados durante el proceso de verificación y validación. Estos elementos fueron seleccionados por su relevancia dentro del funcionamiento del prototipo y por su relación directa con los objetivos del sistema: autenticidad, traza-

bilidad, reventa y verificación de tickets.

Dado que SecureTicket utiliza una arquitectura híbrida, los ítems de prueba incluyen tanto componentes tradicionales de software, como frontend, backend y base de datos, como componentes asociados a la lógica blockchain, tales como contratos inteligentes Soroban, integración con Stellar/Soroban Testnet y firma de operaciones mediante Freighter Wallet.

Nombre	Tipo	Descripción	Objetivo de prueba
Contrato <i>FabricaBoletos</i>	Contrato inteligente	Contrato Soroban encargado de gestionar o desplegar contratos asociados a eventos, según el modelo de arquitectura definido para el sistema.	Verificar que la creación y administración de contratos de evento se realice correctamente bajo las reglas definidas.
Contrato <i>ContratoEvento</i>	Contrato inteligente	Contrato Soroban que contiene la lógica principal relacionada con emisión, reventa, cancelación, invalidación y redención de tickets.	Validar que las operaciones críticas sobre tickets se ejecuten correctamente y que los casos inválidos sean rechazados.
Representación NFT del ticket	Activo digital	Elemento lógico asociado a la representación del ticket como activo digital único, incluyendo identificadores, versión, propietario y estado.	Verificar la unicidad, trazabilidad y relación del ticket con su propietario y ciclo de vida.
Backend/API	Servicio backend	Servicio responsable de exponer endpoints REST, aplicar reglas de negocio, autenticar usuarios, validar roles, procesar operaciones sobre tickets y coordinar la comunicación con persistencia e integraciones.	Verificar el correcto funcionamiento de las reglas de negocio, autorización, validaciones de estado y respuestas ante errores esperados.
Indexador Soroban	Servicio de sincronización	Componente encargado de procesar eventos o estados derivados de la lógica on-chain y reflejarlos en la base de datos off-chain.	Validar la sincronización entre eventos blockchain y PostgreSQL, evitando duplicaciones o inconsistencias visibles.
Base de datos PostgreSQL	Persistencia	Base de datos relacional utilizada para almacenar usuarios, eventos, tickets, órdenes, estados, check-ins y campos asociados a la integración Web3.	Verificar la consistencia de los datos persistidos después de operaciones de compra, reventa, verificación, cancelación e invalidación.

Nombre	Tipo	Descripción	Objetivo de prueba
Frontend web	Interfaz gráfica	Aplicación web utilizada por compradores, vendedores, administradores y personal de verificación para interactuar con el sistema.	Validar que los usuarios puedan ejecutar los flujos principales del prototipo desde la interfaz, incluyendo compra primaria simulada, inventario y escáner QR.
Escáner QR	Flujo funcional / Interfaz de verificación	Funcionalidad utilizada por personal autorizado para verificar tickets mediante códigos QR. En el prototipo operativo, el escáner funciona bajo un enfoque DB-first.	Comprobar que el escáner acepte tickets válidos y rechace tickets usados, inválidos, inexistentes, expirados o procesados por usuarios no autorizados.
Módulo de autenticación y autorización	Servicio de seguridad	Mecanismo encargado de validar identidad, tokens, roles y permisos para acceder a operaciones protegidas del sistema.	Verificar que únicamente los usuarios autorizados puedan ejecutar acciones críticas según su rol y propiedad sobre los tickets.
Flujo de compra primaria simulada	Flujo funcional	Proceso mediante el cual un usuario adquiere un ticket dentro del prototipo, generando una orden y reflejando el ticket en su inventario.	Validar que la compra primaria simulada funcione correctamente sin afirmar operación con pagos reales ni transacción on-chain cuando no corresponda.
Flujo de reventa en Stellar/Soroban Testnet	Flujo funcional / Integración blockchain	Proceso mediante el cual un propietario lista un ticket para reventa y otro usuario lo adquiere utilizando Freightier sobre Stellar/Soroban Testnet.	Validar el flujo central de reventa, verificando cambio de propietario, trazabilidad, actualización de estado y relación con la lógica blockchain del prototipo.
Integración con Freightier Wallet	Integración externa	Extensión de wallet utilizada para firmar operaciones blockchain por parte de usuarios en Stellar/Soroban Testnet.	Verificar manualmente que los flujos que requieren firma de usuario puedan ejecutarse en Testnet sin usar mainnet ni activos reales.

Nombre	Tipo	Descripción	Objetivo de prueba
Integración con Stellar/Soroban Testnet	Integración externa	Red de pruebas utilizada para desplegar o ejecutar operaciones blockchain dentro del alcance académico del sistema.	Validar operaciones blockchain en un ambiente de prueba, excluyendo mainnet, activos con valor real y operación productiva.
Pruebas de carga sobre API REST	Escenario de rendimiento	Conjunto de escenarios orientados a evaluar endpoints de lectura pública, usuarios autenticados y operaciones críticas controladas.	Evaluar estabilidad y tiempos de respuesta del backend bajo carga moderada, sin ejecutar carga masiva sobre Freightier o transacciones Soroban reales.
Pruebas de aceptación de usuario	Validación con usuarios	Actividades guiadas para que usuarios o stakeholders ejecuten tareas representativas y evalúen claridad, comprensión y confianza del sistema.	Obtener retroalimentación sobre la usabilidad básica y comprensión de los flujos principales del prototipo.
Smoke test de despliegue	Validación de ambiente	Prueba de humo orientada a verificar que el frontend, backend y flujos mínimos funcionen en el ambiente desplegado.	Confirmar la disponibilidad básica del prototipo fuera del ambiente local de desarrollo.
Matriz de trazabilidad	Artefacto documental	Relación entre pruebas, requerimientos, casos de uso y atributos de calidad definidos para el sistema.	Asegurar que las pruebas ejecutadas se encuentren asociadas a los elementos funcionales y no funcionales que buscan validar.

Tabla 3: Ítems de prueba definidos para Secure Ticket

Los ítems de prueba descritos en esta sección delimitan el alcance de la estrategia de validación. Aquellos elementos que involucren mainnet, pagos reales, activos con valor económico, procesos KYC/AML o integraciones productivas quedan excluidos del presente plan por encontrarse fuera del alcance académico del prototipo.

4. Alcance de las pruebas

El alcance de las pruebas de SecureTicket comprende la verificación y validación de los componentes principales del prototipo académico, así como de los flujos funcionales que soportan la autenticidad, trazabilidad, reventa y verificación de tickets. Las pruebas se enfocan en evaluar el comportamiento del sistema en ambientes controlados de desarrollo, integración, despliegue académico y Stellar/Soroban Testnet.

Este plan no busca certificar el sistema para operación productiva, sino demostrar que el prototipo implementa correctamente los mecanismos definidos dentro del alcance del trabajo de grado.

4.1. Componentes incluidos en el alcance

Las pruebas cubren los siguientes componentes del sistema:

- **Contratos inteligentes Soroban:** se validan las funciones relacionadas con emisión, compra, reventa, cancelación, invalidación, redención, permisos de verificadores y casos negativos sobre tickets.
- **Backend/API:** se evalúan los endpoints REST, reglas de negocio, autenticación, autorización por roles, validaciones de estado, escáner QR, compra primaria simulada, reventa, cancelación e invalidación según el alcance implementado.
- **Frontend web:** se prueban los flujos principales de usuario disponibles en la interfaz, incluyendo inicio de sesión, consulta de eventos, compra primaria simulada, inventario de tickets y escáner QR.
- **Base de datos PostgreSQL:** se valida la persistencia y consistencia de entidades como usuarios, eventos, tickets, órdenes, estados, registros de verificación y campos asociados a la integración Web3.
- **Indexador o sincronización on-chain/off-chain:** se prueban los mecanismos encargados de reflejar eventos o estados asociados a la lógica blockchain en la base de datos off-chain.
- **Integración con Stellar/Soroban Testnet:** se valida la lógica blockchain en red de pruebas, principalmente para flujos relacionados con contratos inteligentes y reventa manual guiada.

- **Integración con Freightier Wallet:** se contempla dentro del flujo manual guiado de reventa en Testnet, debido a que la automatización de extensiones de wallet puede ser frágil y poco estable.
- **Pruebas de carga sobre API REST:** se incluyen escenarios sobre lectura pública, usuarios autenticados y operaciones críticas controladas.
- **Pruebas de aceptación:** se incluye la validación académica con usuarios o stakeholders sobre tareas representativas del sistema.
- **Smoke test de despliegue:** se valida que los servicios principales del prototipo funcionen en el ambiente desplegado.

4.2. Flujos funcionales incluidos

Los flujos funcionales cubiertos por este plan de pruebas son los siguientes:

Flujo	Descripción	Tipo de prueba asociada
Compra primaria simulada	Flujo mediante el cual un usuario selecciona un evento, realiza un checkout simulado y recibe un ticket en su inventario.	E2E / Backend / Frontend
Reventa en Testnet	Flujo mediante el cual un usuario propietario lista un ticket para reventa y otro usuario lo adquiere usando Freightier sobre Stellar/Soroban Testnet.	Manual guiada / Blockchain
Escáner QR	Flujo mediante el cual un usuario autorizado verifica un ticket mediante código QR, registra el uso del ticket y rechaza casos inválidos.	API / E2E / Seguridad
Redención on-chain	Validación de la función de redención del ticket en el contrato inteligente Soroban.	Contrato inteligente
Invalidación de boleto	Validación de que un ticket invalidado no pueda listarse, comprarse, escanearse o redimirse según el nivel implementado.	Contrato / Backend / API
Cancelación de reventa	Validación de que un ticket listado pueda retirarse del mercado secundario y no pueda ser comprado posteriormente.	Contrato / Backend / API
Autenticación y autorización	Validación de tokens, roles, permisos, propiedad de tickets y operaciones no autorizadas.	Seguridad / API
Sincronización on-chain/off-chain	Validación de que los eventos o estados relevantes se reflejen correctamente en PostgreSQL.	Integración
Carga sobre API REST	Evaluación del comportamiento del backend bajo carga moderada en endpoints representativos.	Rendimiento
Aceptación de usuario	Validación de que usuarios o stakeholders puedan completar tareas representativas y comprender los estados principales del sistema.	Aceptación / Formulario

Tabla 4: Flujos funcionales incluidos en el alcance de pruebas

4.3. Niveles de prueba incluidos

El plan contempla los siguientes niveles de prueba:

- **Pruebas unitarias:** orientadas a validar componentes aislados, principalmente contratos inteligentes y lógica de backend.
- **Pruebas de integración:** enfocadas en la interacción entre backend, base de datos, indexador y eventos asociados a la lógica on-chain.
- **Pruebas de API:** dirigidas a verificar endpoints, códigos de respuesta, validaciones, manejo de errores y reglas de autorización.
- **Pruebas end-to-end:** orientadas a validar flujos principales desde la perspectiva del usuario a través de la interfaz web.
- **Pruebas manuales guiadas:** utilizadas para flujos donde la automatización no es conveniente o estable, especialmente la reventa con Freightier sobre Stellar/Soroban Testnet.
- **Pruebas de carga:** enfocadas en medir el comportamiento de endpoints REST bajo carga moderada.
- **Pruebas de aceptación:** orientadas a obtener retroalimentación de usuarios o stakeholders sobre tareas representativas del prototipo.
- **Pruebas documentales:** utilizadas para mantener trazabilidad entre requerimientos, casos de uso, atributos de calidad y pruebas ejecutadas.

4.4. Cobertura parcial por diseño

Algunas funcionalidades se consideran parcialmente cubiertas debido al alcance real del prototipo y a decisiones arquitectónicas del sistema. Estas condiciones se documentan explícitamente para evitar afirmar capacidades que no forman parte de la implementación actual.

Elemento	Cobertura actual	Justificación
Escáner QR con redención on-chain	El escáner operativo se valida bajo un enfoque DB-first; la redención on-chain se prueba a nivel de contrato.	La integración directa de redención on-chain en el escáner no hace parte del flujo operativo actual del prototipo.
Invalidación administrativa por API	La invalidación se cubre mediante contrato, reglas backend y validaciones en flujos de escáner, listado y compra.	No existe un endpoint administrativo completo de invalidación en la versión actual del prototipo.
Reventa con Freightier	La reventa se valida mediante una prueba manual guiada sobre Stellar/Soroban Testnet.	Automatizar Freightier puede generar pruebas frágiles y poco mantenibles.
Compra primaria	La compra primaria se valida como flujo simulado/off-chain del prototipo.	El uso de pagos reales o transacciones productivas queda fuera del alcance académico.
Carga blockchain	No se ejecuta carga masiva sobre Freightier ni transacciones Soroban reales.	Las pruebas de carga se limitan a endpoints REST representativos para evitar resultados poco representativos o riesgos innecesarios sobre Testnet.

Tabla 5: Elementos con cobertura parcial por diseño

4.5. Elementos fuera del alcance

Quedan fuera del alcance de este plan de pruebas los siguientes elementos:

- Despliegue o validación sobre Stellar Mainnet.
- Uso de activos con valor real.
- Integración con pagos reales o pasarelas de pago productivas.
- Validaciones comerciales, legales o financieras asociadas a operación productiva.
- Procesos KYC/AML.
- Pruebas de penetración formales o auditorías externas de seguridad.
- Pruebas de carga masiva sobre transacciones blockchain reales.
- Automatización completa de flujos con Freightier Wallet.
- Pruebas sobre aplicaciones móviles nativas.
- Certificación de disponibilidad, escalabilidad o tolerancia a fallos para producción.

4.6. Criterio de delimitación del alcance

Una prueba se considera dentro del alcance cuando evalúa una funcionalidad implementada en el prototipo, puede ejecutarse en ambiente local, staging, despliegue académico o Stellar/Soroban Testnet, y contribuye a validar uno de los objetivos principales del sistema: autenticidad, trazabilidad, reventa o verificación de tickets.

Por el contrario, una prueba se considera fuera del alcance cuando requiere operación en mainnet, activos reales, pagos productivos, infraestructura comercial, validaciones legales especializadas o capacidades no implementadas en la versión actual del sistema.

5. Estrategia de pruebas

La estrategia de pruebas de SecureTicket se define a partir de una combinación de pruebas automatizadas, pruebas de integración, pruebas end-to-end, pruebas manuales guiadas, pruebas de carga y pruebas de aceptación. Esta combinación responde a la naturaleza híbrida del sistema, el cual integra frontend web, backend/API, base de datos PostgreSQL, contratos inteligentes Soroban y flujos sobre Stellar/Soroban Testnet.

El enfoque general consiste en validar primero los componentes de forma aislada, posteriormente sus integraciones principales y finalmente los flujos funcionales que representan el comportamiento esperado del prototipo desde la perspectiva del usuario. De esta forma, las pruebas se organizan en niveles progresivos: contratos, backend, integración, frontend, flujos end-to-end, carga, despliegue y aceptación.

Todas las pruebas relacionadas con blockchain se ejecutan en entornos locales de contrato o en Stellar/Soroban Testnet. No se contempla el uso de mainnet, activos reales ni pagos productivos, debido a que estos elementos se encuentran fuera del alcance académico del proyecto.

5.1. Pruebas unitarias de contratos inteligentes

Las pruebas unitarias de contratos inteligentes tienen como objetivo validar la lógica on-chain implementada en Soroban. Estas pruebas permiten verificar el comportamiento de las funciones críticas de los contratos sin depender de la interfaz gráfica ni de flujos externos del sistema.

En este nivel se validan operaciones como emisión de tickets, reventa, cancelación, inva-

lidación, redención, asignación de verificadores autorizados, control de estados y rechazo de casos inválidos.

- **Herramientas:** `cargo test`, entorno de pruebas de Rust y Soroban SDK.
- **Ambiente:** ambiente local de desarrollo para contratos inteligentes.
- **Ítems evaluados:** contratos *FabricaBoletos*, *ContratoEvento* y componentes asociados a la representación del ticket.
- **Criterios de completitud:**
 - Ejecución exitosa de las pruebas de contrato.
 - Cobertura de emisión, reventa, cancelación, invalidación y redención.
 - Inclusión de casos negativos para usuarios no autorizados, tickets inexistentes, tickets usados o tickets inválidos.
 - Validación de que los estados del ticket impidan operaciones no permitidas.

5.2. Pruebas unitarias y de API del backend

Las pruebas del backend permiten validar la lógica de negocio expuesta por la API, incluyendo autenticación, autorización, reglas de operación sobre tickets, escáner QR, cancelación, validaciones de estado y manejo de errores esperados.

Estas pruebas verifican que el backend aplique restricciones de seguridad y reglas de negocio de forma independiente al frontend, evitando que operaciones críticas dependan únicamente de validaciones visuales o del cliente.

- **Herramientas:** framework de pruebas del backend, Supertest o herramientas equivalentes utilizadas en el repositorio.
- **Ambiente:** ambiente local de desarrollo e integración con base de datos de prueba.
- **Ítems evaluados:** endpoints REST, servicios de autenticación, autorización, escáner QR, operaciones sobre tickets, reglas de reventa e invalidación según el alcance implementado.
- **Criterios de completitud:**

- Ejecución exitosa de las pruebas unitarias y de API.
- Validación de respuestas esperadas ante tokens inválidos, expirados o ausentes.
- Validación de restricciones por roles *CUSTOMER*, *STAFF* y *ADMIN*.
- Rechazo de operaciones sobre tickets ajenos, usados, inexistentes o inválidos.
- Manejo controlado de errores de negocio y solicitudes manipuladas.

5.3. Pruebas de integración

Las pruebas de integración tienen como propósito validar la interacción entre componentes del sistema que no pueden evaluarse de forma completamente aislada. En SecureTicket, estas pruebas se enfocan principalmente en la relación entre backend, base de datos PostgreSQL, indexador y eventos asociados a la lógica on-chain.

El objetivo es comprobar que los cambios derivados de operaciones sobre tickets se reflejen de forma consistente en la persistencia off-chain, evitando duplicaciones, pérdida de eventos o estados inconsistentes.

- **Herramientas:** framework de pruebas del backend, base de datos de prueba, fixtures o eventos normalizados.
- **Ambiente:** ambiente local de integración con PostgreSQL.
- **Ítems evaluados:** indexador Soroban, sincronización on-chain/off-chain, persistencia de tickets, estados y campos Web3.
- **Criterios de completitud:**
 - Procesamiento correcto de eventos de boleto creado, listado, cancelado, revendido, redimido e invalidado.
 - Actualización correcta de campos como propietario, versión, estado de venta y precio de reventa.
 - Manejo controlado de eventos desconocidos o mal formados.
 - Idempotencia ante eventos duplicados.
 - Ausencia de inconsistencias visibles entre el estado esperado y el estado persistido.

5.4. Pruebas end-to-end del frontend

Las pruebas end-to-end permiten validar flujos completos desde la perspectiva del usuario mediante la interfaz web. En este plan se priorizan los flujos principales que representan el uso esperado del prototipo: inicio de sesión, consulta de eventos, compra primaria simulada, inventario y escáner QR.

Estas pruebas no buscan cubrir exhaustivamente todos los estados visuales de la aplicación, sino comprobar que los usuarios puedan completar tareas funcionales críticas utilizando la interfaz.

- **Herramientas:** Playwright o herramienta E2E equivalente configurada en el frontend.
- **Ambiente:** ambiente local o despliegue académico con frontend y backend disponibles.
- **Ítems evaluados:** frontend web, navegación, compra primaria simulada, inventario, escáner QR y mensajes de error.
- **Criterios de completitud:**
 - El usuario puede iniciar sesión y navegar por eventos.
 - El usuario puede ejecutar una compra primaria simulada y visualizar el ticket en inventario.
 - El escáner QR puede mostrar resultados de éxito y rechazo según el caso evaluado.
 - Los usuarios no autorizados son bloqueados en flujos restringidos.
 - Los errores principales se presentan de forma comprensible para el usuario.

5.5. Prueba manual guiada de reventa en Testnet

La reventa con Freighter Wallet se define como una prueba manual guiada debido a que la automatización de extensiones de wallet puede producir pruebas frágiles y difíciles de mantener. Esta prueba permite validar el flujo central del sistema sobre Stellar/Soroban Testnet sin involucrar mainnet ni activos reales.

El flujo contempla que un usuario propietario liste un ticket para reventa y que un segundo usuario lo compre mediante una operación firmada con Freighter. Se verifican el cambio de propietario, la actualización de estado, la trazabilidad del ticket y la consistencia con PostgreSQL.

- **Herramientas:** navegador web, Freighter Wallet, frontend del sistema, backend/API, PostgreSQL y Stellar/Soroban Testnet.
- **Ambiente:** ambiente local o despliegue académico configurado contra Stellar/Soroban Testnet.
- **Ítems evaluados:** flujo de reventa, integración con Freighter, contrato en Testnet, marketplace, inventario y persistencia.
- **Criterios de completitud:**
 - Usuario propietario puede listar un ticket para reventa.
 - Un segundo usuario puede comprar el ticket listado utilizando Freighter.
 - La operación se realiza sobre Stellar/Soroban Testnet.
 - Se verifica cambio de propietario y actualización del inventario.
 - Se valida que el ticket no permanezca disponible indebidamente para el propietario anterior.
 - Se prueban casos negativos como compra del propio ticket, listado de ticket ajeno o rechazo de firma.

5.6. Pruebas del escáner QR

Las pruebas del escáner QR se enfocan en validar el flujo de verificación de tickets desde la perspectiva operativa del prototipo. El escáner funciona bajo un enfoque DB-first, por lo cual la validación de uso del ticket y el registro de verificación se realizan principalmente contra la base de datos del sistema.

La redención on-chain se evalúa de forma separada mediante pruebas de contrato inteligente, sin afirmar que el escáner operativo ejecute redención directamente sobre Soroban.

- **Herramientas:** pruebas de API, pruebas E2E y frontend del escáner.
- **Ambiente:** ambiente local o despliegue académico.

- **Ítems evaluados:** escáner QR, control de acceso por rol, registros de verificación, estados de ticket y manejo de códigos QR inválidos.
- **Criterios de completitud:**
 - El escáner acepta tickets válidos con usuarios autorizados.
 - El sistema rechaza códigos QR falsos, expirados, desactualizados, inexistentes, usados o asociados a tickets inválidos.
 - Se impide el doble escaneo de un mismo ticket.
 - Se restringe el acceso al escáner para usuarios no autorizados.
 - El flujo crea o actualiza los registros de verificación de forma consistente.

5.7. Pruebas de seguridad funcional

Las pruebas de seguridad funcional buscan verificar que las acciones críticas del sistema estén protegidas mediante autenticación, autorización por roles, validación de propiedad del ticket y validación de solicitudes manipuladas.

Estas pruebas no corresponden a una auditoría de seguridad formal ni a un pentest, sino a la validación de controles funcionales esperados para el prototipo.

- **Herramientas:** pruebas de API, Supertest o herramienta equivalente.
- **Ambiente:** ambiente local de pruebas.
- **Ítems evaluados:** autenticación JWT, roles, endpoints protegidos, propiedad de tickets, wallet asociada y validaciones de solicitud.
- **Criterios de completitud:**
 - Rechazo de solicitudes sin token, con token inválido o token expirado.
 - Rechazo de usuarios con rol insuficiente en endpoints protegidos.
 - Rechazo de operaciones sobre tickets ajenos.
 - Rechazo de manipulación de precio, estado o identificadores desde el request.
 - Validación de que las restricciones se apliquen en backend y no únicamente en frontend.

5.8. Pruebas de carga

Las pruebas de carga buscan evaluar el comportamiento del backend/API bajo una carga moderada y representativa del prototipo. Estas pruebas se limitan a endpoints REST y no incluyen carga masiva sobre Freightier, mainnet, pagos reales ni transacciones Soroban reales.

La estrategia contempla tres escenarios: lectura pública, usuarios autenticados y operaciones críticas controladas.

- **Herramientas:** k6 o herramienta equivalente.
- **Ambiente:** ambiente local o despliegue académico.
- **Ítems evaluados:** endpoints de salud, eventos, marketplace, login, inventario, escáner y checkout simulado cuando aplique.
- **Criterios de completitud:**
 - Ejecución de un escenario de lectura pública con carga moderada.
 - Ejecución de un escenario de usuarios autenticados.
 - Ejecución de un escenario de operaciones críticas controladas.
 - Tasa de error HTTP menor o igual al 1 % de las solicitudes ejecutadas.
 - Tiempo de respuesta p95 menor o igual a 1500 ms para endpoints de lectura pública.
 - Tiempo de respuesta p95 menor o igual a 2500 ms para endpoints autenticados u operaciones críticas controladas.
 - Separación explícita entre pruebas REST y operaciones blockchain.

5.9. Pruebas de humo de despliegue

Las pruebas de humo de despliegue permiten verificar que el sistema funcione de manera básica en el ambiente desplegado. Su propósito no es realizar una validación exhaustiva, sino confirmar que los servicios principales están disponibles y que los flujos mínimos pueden ejecutarse.

- **Herramientas:** navegador web, endpoints de salud, frontend desplegado y backend desplegado.

- **Ambiente:** ambiente de despliegue académico.
- **Ítems evaluados:** frontend, backend, autenticación, eventos, marketplace, inventario y escáner.
- **Criterios de completitud:**
 - El frontend carga correctamente.
 - El backend responde en el endpoint de salud.
 - El usuario puede iniciar sesión.
 - Los eventos y marketplace se consultan correctamente.
 - El inventario y escáner funcionan según el rol correspondiente.
 - No se presentan errores críticos visibles en consola o backend durante el flujo mínimo.

5.10. Pruebas de aceptación de usuario

Las pruebas de aceptación de usuario permiten obtener una validación académica controlada sobre la comprensión y ejecución de tareas representativas del sistema. Estas pruebas se orientan a usuarios o stakeholders que interactúan con el prototipo desde roles como comprador, vendedor, staff o administrador.

- **Herramientas:** formulario de aceptación, guía de tareas y ambiente del prototipo.
- **Ambiente:** ambiente local, staging o despliegue académico.
- **Ítems evaluados:** claridad del flujo, comprensión del estado del ticket, confianza percibida sobre autenticidad y capacidad de completar tareas.
- **Criterios de completitud:**
 - Participación de usuarios o stakeholders en tareas representativas.
 - Registro de resultados mediante formulario con escala de valoración.
 - Identificación de errores, confusiones o sugerencias.
 - Consolidación de resultados por rol y tarea.
 - Análisis de la percepción de aceptación del prototipo dentro del alcance académico.

5.11. Trazabilidad de pruebas

La trazabilidad permite relacionar cada prueba con los requerimientos, casos de uso o atributos de calidad que busca validar. Esta actividad permite justificar la cobertura del plan de pruebas y facilita identificar funcionalidades cubiertas, parcialmente cubiertas o fuera del alcance.

- **Herramientas:** matriz de trazabilidad.
- **Ambiente:** artefacto documental del proyecto.
- **Ítems evaluados:** relación entre pruebas, requerimientos, casos de uso, atributos de calidad y estado de ejecución.
- **Criterios de completitud:**
 - Cada prueba crítica cuenta con identificador único.
 - Cada prueba se asocia con al menos un requerimiento, caso de uso o atributo de calidad.
 - La matriz diferencia estado de implementación y estado de ejecución.
 - Las coberturas parciales y exclusiones por alcance quedan documentadas.

6. Ambiente y datos de prueba

Esta sección describe los ambientes, herramientas, configuraciones y datos utilizados para ejecutar las pruebas de SecureTicket. Dado que el sistema corresponde a un prototipo académico, las pruebas se realizan en ambientes controlados y no contemplan operación sobre mainnet, pagos reales ni activos con valor económico.

El ambiente de pruebas se organiza en cuatro niveles principales: ambiente local de desarrollo, ambiente de integración, ambiente de despliegue académico y ambiente blockchain de pruebas sobre Stellar/Soroban Testnet. Esta separación permite validar los componentes de manera aislada y posteriormente verificar los flujos principales del sistema de forma integrada.

6.1. Ambientes de prueba

Los ambientes considerados para la ejecución de pruebas son los siguientes:

Ambiente	Descripción	Uso dentro del plan de pruebas
Ambiente local de desarrollo	Entorno ejecutado en las máquinas del equipo de desarrollo, con frontend, backend, base de datos y contratos configurados localmente o contra servicios de prueba.	Ejecución de pruebas unitarias, pruebas de API, pruebas de integración, pruebas E2E locales y validaciones iniciales de flujos.
Ambiente de integración	Entorno utilizado para validar la interacción entre backend, base de datos, indexador y eventos normalizados asociados a la lógica on-chain.	Ejecución de pruebas de integración, sincronización on-chain/off-chain y validación de persistencia.
Ambiente de despliegue académico	Entorno desplegado para validar que el prototipo pueda ejecutarse fuera del ambiente local, incluyendo frontend, backend y servicios asociados.	Ejecución de smoke test de despliegue, pruebas de aceptación y validación básica de disponibilidad.
Stellar/Soroban Testnet	Red de pruebas utilizada para validar operaciones blockchain sin activos reales ni exposición a mainnet.	Ejecución de pruebas relacionadas con contratos inteligentes, reventa manual guiada con Freightier y validación de transacciones de prueba.

Tabla 6: Ambientes definidos para la ejecución de pruebas

6.2. Configuración técnica del ambiente

La configuración técnica contempla los componentes principales requeridos para ejecutar el prototipo y sus pruebas.

Componente	Tecnología / herramienta	Propósito en pruebas
Frontend web	React, Vite, TypeScript y Playwright	Ejecutar la interfaz del usuario y validar flujos end-to-end del prototipo.
Backend/API	Node.js, Express, TypeScript, framework de pruebas del backend y Supertest	Ejecutar reglas de negocio, endpoints REST, pruebas unitarias, pruebas de API y validaciones de seguridad funcional.
Contratos inteligentes	Rust, Soroban SDK y cargo test	Validar la lógica on-chain de emisión, reventa, cancelación, invalidación y redención de tickets.
Base de datos	PostgreSQL	Persistir usuarios, eventos, tickets, órdenes, registros de verificación y campos asociados a la integración Web3.
ORM / migraciones	Prisma o herramienta equivalente definida en el repositorio	Gestionar el esquema de base de datos y validar el estado de migraciones.
Indexador	Servicio de sincronización implementado en backend	Procesar eventos o estados asociados a la lógica on-chain y reflejarlos en PostgreSQL.
Wallet	Freighter Wallet	Firmar operaciones de usuario en Stellar/Soroban Testnet durante la prueba manual guiada de reventa.
Red blockchain	Stellar/Soroban Testnet	Ejecutar operaciones blockchain de prueba sin usar mainnet ni activos reales.
Pruebas de carga	k6	Ejecutar escenarios de carga sobre endpoints REST representativos.
Pruebas de aceptación	Formulario de aceptación y guía de tareas	Recoger retroalimentación de usuarios o stakeholders sobre los flujos principales del prototipo.

Tabla 7: Configuración técnica del ambiente de pruebas

6.3. Variables de entorno

Para ejecutar las pruebas se requiere configurar variables de entorno asociadas al frontend, backend, base de datos, autenticación, contratos y red Stellar/Soroban Testnet. Los valores sensibles no deben incluirse en este documento ni en el repositorio público; únicamente se documentan los nombres de variables y su propósito.

Variable	Propósito	Componente
DATABASE_URL	Cadena de conexión a PostgreSQL para ejecución del backend y pruebas de integración.	Backend / BD
JWT_SECRET	Secreto utilizado para firmar o validar tokens de autenticación en ambiente de prueba.	Backend
SOROBAN_RPC_URL	URL del endpoint RPC de Stellar/Soroban Testnet.	Backend / Blockchain
STELLAR_NETWORK	Identificador de la red Stellar utilizada. Para pruebas debe corresponder a Testnet.	Backend / Blockchain
CONTRACT_ADDRESS	Dirección del contrato utilizado en pruebas o flujos de integración cuando aplique.	Backend / Blockchain
FACTORY_CONTRACT_ADDRESS	Dirección del contrato Factory cuando el flujo lo requiera.	Backend / Blockchain
FRONTEND_URL	URL del frontend en ambiente local o desplegado.	Frontend / E2E
BACKEND_URL	URL del backend/API utilizada por pruebas E2E, smoke test o carga.	Backend / E2E
BASE_URL	URL base usada por los scripts de carga k6.	Carga
E2E_REAL	Bandera para habilitar pruebas E2E contra ambiente real cuando aplique.	Frontend / E2E

Tabla 8: Variables de entorno requeridas para pruebas

Los valores concretos de estas variables deben definirse en archivos locales de configuración, por ejemplo `.env`, `.env.test` o `.env.example`, según la estructura del repositorio. Ningún secreto real debe incluirse dentro del documento de pruebas.

6.4. Datos de prueba

Los datos de prueba se definen para representar los actores, eventos, tickets y estados necesarios para ejecutar los escenarios funcionales del sistema. Estos datos pueden provenir de seed data, fixtures, registros creados durante la prueba o configuraciones manuales del ambiente.

Tipo de dato	Descripción	Uso en pruebas
Usuarios compradores	Cuentas con rol <i>CUSTOMER</i> utilizadas para compra primaria simulada, inventario y compra de reventa.	Pruebas E2E, API, aceptación y reventa manual guiada.
Usuario vendedor	Cuenta con rol <i>CUSTOMER</i> que posee tickets disponibles para listar en reventa.	Prueba manual guiada de reventa y validación de marketplace.
Usuario staff	Cuenta con rol <i>STAFF</i> autorizada para operar el escáner QR.	Pruebas de escáner, seguridad por roles y smoke test.
Usuario administrador	Cuenta con rol <i>ADMIN</i> utilizada para validar operaciones administrativas permitidas.	Pruebas de seguridad, escáner, smoke test y flujos administrativos.
Wallets de prueba	Cuentas Freighter o llaves de Stellar Testnet asociadas a usuarios de prueba.	Reventa manual guiada, firma de transacciones y validación de propietario.
Eventos de prueba	Eventos registrados en el sistema con tickets disponibles o asociados a contratos.	Compra primaria, inventario, marketplace y reventa.
Tickets disponibles	Tickets activos y operables asociados a eventos de prueba.	Compra, reventa, inventario y escáner.
Tickets usados	Tickets marcados como usados o con registro de verificación asociado.	Validación de rechazo de doble escaneo y operaciones no permitidas.

Tabla 9: Datos de prueba funcionales definidos para los escenarios principales

Tipo de dato	Descripción	Uso en pruebas
Tickets inválidos o invalidables	Tickets utilizados para validar restricciones de invalidación, compra, listado o escaneo.	Pruebas de contrato, API y escáner.
Tickets listados en reventa	Tickets marcados como disponibles en marketplace secundario.	Reventa manual guiada, cancelación y validación de marketplace.
Códigos QR válidos	Representaciones QR asociadas a tickets existentes y activos.	Escáner QR y pruebas E2E de verificación.
Códigos QR inválidos	Códigos QR falsos, expirados, desactualizados, inexistentes o manipulados.	Casos negativos del escáner QR.
Eventos on-chain normalizados	Fixtures o estructuras simuladas de eventos emitidos por contratos Soroban.	Pruebas de integración del indexador y sincronización on-chain/off-chain.

Tabla 10: Datos de prueba negativos y de integración

6.5. Estados de ticket considerados

Las pruebas consideran diferentes estados del ticket para validar que el sistema permita o rechace operaciones según corresponda. La nomenclatura exacta puede variar de acuerdo con la implementación, pero conceptualmente se manejan los siguientes estados:

Estado	Descripción	Uso en pruebas
Activo / Disponible	Ticket válido que puede ser usado, comprado o listado según el flujo correspondiente.	Compra, reventa e inventario.
Listado en reventa	Ticket ofrecido en el marketplace secundario por su propietario.	Reventa y cancelación de reventa.
Vendido / Asignado	Ticket asociado a un comprador después de una compra primaria simulada o reventa.	Inventario y trazabilidad.
Usado	Ticket que ya fue verificado mediante escáner o redimido según el nivel evaluado.	Rechazo de doble escaneo y prevención de reutilización.
Invalidado	Ticket marcado como no operable por reglas del sistema o contrato.	Rechazo de compra, listado, escaneo o redención.
Cancelado de reventa	Ticket retirado del marketplace secundario por su propietario o por regla del sistema.	Validación de cancelación y rechazo de compra posterior.
Inexistente o manipulado	Identificador de ticket que no corresponde a un registro válido del sistema.	Casos negativos de API, escáner y seguridad.

Tabla 11: Estados de ticket considerados durante las pruebas

6.6. Datos para pruebas de carga

Las pruebas de carga requieren datos estables y no destructivos que permitan ejecutar escenarios repetibles sobre endpoints REST. Para evitar alteración no controlada del estado del sistema, se priorizan endpoints de lectura y operaciones controladas.

Escenario	Datos requeridos	Observaciones
Lectura pública	Eventos existentes, marketplace o listado público de tickets.	No requiere autenticación ni modifica datos críticos.
Usuarios autenticados	Usuarios de prueba con credenciales válidas y tickets asociados.	Requiere tokens o credenciales configuradas mediante variables de entorno.
Operaciones críticas controladas	Usuario staff/admin, código QR de prueba, ticket controlado o flujo de checkout simulado.	Debe evitar datos destructivos o reiniciar el estado antes de cada ejecución.

Tabla 12: Datos requeridos para pruebas de carga

6.7. Consideraciones de seguridad sobre datos de prueba

Los datos utilizados durante las pruebas deben cumplir las siguientes condiciones:

- No deben incluir información personal sensible real.
- No deben utilizar activos con valor económico.

- No deben incluir claves privadas reales dentro del repositorio o del documento.
- Las wallets utilizadas deben corresponder exclusivamente a Stellar/Soroban Testnet.
- Las credenciales de prueba deben poder rotarse o reemplazarse sin afectar el sistema.
- Los códigos QR utilizados para pruebas negativas deben ser generados únicamente con fines de validación.
- Los datos destructivos o de cambio de estado deben aislarse para evitar afectar otros escenarios de prueba.

6.8. Criterios de preparación del ambiente

Antes de ejecutar las pruebas, se debe verificar que el ambiente cumpla con los siguientes criterios:

- El repositorio se encuentra en la rama y commit definidos para la ejecución de pruebas.
- Las dependencias del frontend, backend y contratos están instaladas correctamente.
- Las variables de entorno requeridas están configuradas.
- La base de datos está disponible y con migraciones aplicadas.
- Los usuarios, eventos, tickets y códigos QR de prueba están creados o disponibles.
- Los contratos requeridos están compilados o desplegados según el tipo de prueba.
- La red configurada para blockchain corresponde a Stellar/Soroban Testnet.
- El backend y frontend se encuentran ejecutándose para pruebas E2E, smoke test o carga.
- Freight Wallet está configurado con cuentas de Testnet para la prueba manual guiada de reventa.
- Los scripts de carga y guías manuales se encuentran actualizados en el repositorio.

7. Criterios de entrada y salida

Los criterios de entrada y salida establecen las condiciones mínimas que deben cumplirse antes de iniciar la ejecución de las pruebas y las condiciones necesarias para considerar que una prueba, grupo de pruebas o fase de validación ha finalizado satisfactoriamente. Estos criterios permiten controlar la ejecución del plan de pruebas, reducir ambigüedades y asegurar que los resultados obtenidos sean trazables y reproducibles.

Dado que SecureTicket integra componentes frontend, backend, base de datos, contratos inteligentes y servicios externos de prueba, los criterios se definen tanto de forma general como por tipo de prueba.

7.1. Criterios generales de entrada

Antes de iniciar la ejecución formal de las pruebas, se deben cumplir las siguientes condiciones:

Criterio	Descripción	Estado
Versión congelada	Debe existir una rama, tag o commit definido para la ejecución de pruebas, evitando cambios no controlados durante la validación.	Pendiente
Repositorio disponible	El código fuente del frontend, backend, contratos y pruebas debe estar disponible en el repositorio oficial del proyecto.	Pendiente
Dependencias instaladas	Las dependencias requeridas para frontend, backend, contratos y herramientas de prueba deben estar instaladas correctamente.	Pendiente
Variables de entorno configuradas	Los archivos de configuración requeridos deben estar disponibles en el ambiente de prueba, sin exponer secretos reales.	Pendiente
Base de datos disponible	PostgreSQL debe estar disponible y con las migraciones aplicadas para ejecutar pruebas de API, integración y E2E.	Pendiente
Datos de prueba preparados	Usuarios, eventos, tickets, roles, códigos QR y wallets de Testnet deben estar disponibles según el tipo de prueba.	Pendiente
Contratos compilados	Los contratos inteligentes deben compilar correctamente antes de ejecutar pruebas de contrato o flujos blockchain.	Pendiente
Red blockchain configurada	Las pruebas blockchain deben estar configuradas exclusivamente contra Stellar/Soroban Testnet o entorno local de contrato.	Pendiente
Servicios ejecutándose	Para pruebas E2E, carga, smoke test o aceptación, el frontend y backend deben estar activos y accesibles.	Pendiente
Freighter configurado	Para la prueba manual de reventa, Freighter Wallet debe estar configurado con cuentas de Stellar Testnet y fondos de prueba suficientes.	Pendiente
Matriz de trazabilidad disponible	Debe existir una matriz base que relacione pruebas con requerimientos, casos de uso o atributos de calidad.	Pendiente

Tabla 13: Criterios generales de entrada para la ejecución de pruebas

El estado de cada criterio será actualizado durante la fase de ejecución formal de pruebas.

7.2. Criterios generales de salida

Una vez ejecutadas las pruebas, se considerará que el proceso de validación cumple su salida general cuando se satisfagan las siguientes condiciones:

Criterio	Descripción	Estado
Pruebas ejecutadas	Las pruebas planificadas para el alcance definido deben haberse ejecutado o contar con justificación explícita en caso de no ejecución.	Pendiente
Resultados registrados	Cada prueba ejecutada debe contar con estado: aprobada, fallida, parcial, bloqueada, no ejecutada o no aplica.	Pendiente
Fallos bloqueantes identificados	Los errores críticos encontrados durante la ejecución deben estar documentados y clasificados.	Pendiente
Fallos bloqueantes corregidos o justificados	Los defectos que impidan validar un flujo crítico deben corregirse o declararse como limitación del prototipo.	Pendiente
Evidencia asociada	Las pruebas ejecutadas deben contar con evidencia correspondiente, como logs, resultados de consola, capturas, reportes, hashes de transacción o formularios.	Pendiente
Trazabilidad actualizada	La matriz de trazabilidad debe reflejar el estado real de implementación y ejecución de cada prueba.	Pendiente
Limitaciones documentadas	Las coberturas parciales, exclusiones y decisiones de alcance deben estar explícitamente registradas.	Pendiente
Conclusión de calidad	Debe existir una conclusión sobre el estado de calidad del prototipo con base en los resultados obtenidos.	Pendiente

Tabla 14: Criterios generales de salida del proceso de validación

7.3. Criterios de entrada por tipo de prueba

Además de los criterios generales, cada tipo de prueba requiere condiciones específicas antes de su ejecución.

Tipo de prueba	Criterios de entrada
Pruebas de contratos	Los contratos deben compilar correctamente; el entorno Rust/Soroban debe estar configurado; los casos de prueba deben estar implementados en el repositorio.
Pruebas backend/API	El backend debe tener dependencias instaladas; la base de datos de prueba debe estar disponible; las variables de entorno deben estar configuradas; los usuarios y roles de prueba deben existir o ser creados por fixtures.
Pruebas de integración	La base de datos debe estar migrada; los fixtures o eventos normalizados deben estar disponibles; el servicio de indexación o módulo equivalente debe poder ejecutarse en ambiente de prueba.
Pruebas frontend/E2E	El frontend y backend deben estar ejecutándose; los usuarios de prueba deben existir; los datos mínimos de eventos y tickets deben estar disponibles; la herramienta E2E debe estar configurada.
Prueba manual de reventa Testnet	Freighter debe estar instalado; las wallets de Testnet deben estar configuradas; los usuarios deben estar vinculados a wallets; el ticket a revender debe existir; los contratos y backend deben apuntar a Testnet.
Pruebas del escáner QR	Deben existir tickets válidos, usados, inválidos o inexistentes según el caso; los usuarios STAFF y ADMIN deben estar disponibles; los códigos QR de prueba deben estar generados.
Pruebas de carga	El backend debe estar disponible; los endpoints definidos deben responder correctamente en condiciones normales; las credenciales o tokens de prueba deben estar configurados; los datos usados deben ser estables.
Smoke test de despliegue	Las URLs de frontend y backend deben estar disponibles; el commit desplegado debe estar identificado; los usuarios y datos mínimos deben existir.
Pruebas de aceptación	La guía de tareas debe estar definida; el formulario debe estar preparado; los usuarios o stakeholders deben estar seleccionados; el ambiente debe estar estable para ejecutar las tareas.
Trazabilidad	La matriz debe contener los identificadores de pruebas, su tipo, nivel y relación con requerimientos o casos de uso.

Tabla 15: Criterios de entrada por tipo de prueba

7.4. Criterios de salida por tipo de prueba

Cada grupo de pruebas se considera finalizado cuando cumple los siguientes criterios de salida:

Tipo de prueba	Criterios de salida
Pruebas de contratos	Todas las pruebas ejecutadas deben pasar o, en caso de falla, el defecto debe estar documentado. Deben quedar cubiertos los casos críticos de emisión, reventa, cancelación, invalidación, redención y negativos relevantes.
Pruebas backend/API	Los endpoints críticos deben responder según lo esperado; las reglas de negocio, autorización, escáner y validaciones de estado deben aprobarse sin fallos bloqueantes.
Pruebas de integración	Los eventos o estados procesados deben reflejarse correctamente en PostgreSQL; no debe haber duplicaciones ni inconsistencias visibles en los datos evaluados.
Pruebas frontend/E2E	Los flujos principales deben completarse desde la interfaz o mostrar errores controlados cuando corresponda. Los errores visuales o funcionales críticos deben quedar registrados.
Prueba manual de reventa Testnet	La guía debe ejecutarse completamente o registrar el punto exacto de bloqueo. Debe quedar claro si hubo cambio de propietario, actualización de estado y transacción en Testnet.
Pruebas del escáner QR	El escáner debe aceptar códigos QR válidos y rechazar códigos QR inválidos, usados, expirados, inexistentes o ejecutados por usuarios no autorizados.
Pruebas de carga	Los escenarios deben completar su ejecución y reportar métricas de tasa de error, tiempo promedio, tiempo máximo y percentil 95. Los resultados deben compararse con los umbrales definidos.
Smoke test de despliegue	Los servicios principales deben estar disponibles y permitir ejecutar los flujos mínimos definidos para el ambiente desplegado.
Pruebas de aceptación	Los participantes deben completar las tareas asignadas o registrar los bloqueos encontrados. Los formularios deben consolidarse para análisis.
Trazabilidad	La matriz debe actualizarse con el estado real de cada prueba y su relación con requerimientos, casos de uso o atributos de calidad.

Tabla 16: Criterios de salida por tipo de prueba

7.5. Estados de resultado de prueba

Para unificar el registro de resultados, cada prueba será clasificada con uno de los siguientes estados:

Estado	Descripción
Aprobada	La prueba se ejecutó y el resultado obtenido coincide con el resultado esperado.
Fallida	La prueba se ejecutó, pero el resultado obtenido no coincide con el resultado esperado.
Parcial	La prueba se ejecutó o implementó parcialmente, pero no cubre la totalidad del flujo o depende de una limitación del prototipo.
Bloqueada	La prueba no pudo ejecutarse por ausencia de ambiente, datos, credenciales, dependencia externa o defecto bloqueante.
No ejecutada	La prueba está definida o implementada, pero aún no se ha ejecutado formalmente.
No aplica	La prueba no corresponde al alcance del prototipo o requiere capacidades fuera del alcance académico, como mainnet, pagos reales o infraestructura productiva.

Tabla 17: Estados utilizados para registrar resultados de prueba

7.6. Criterio de suspensión y reanudación

Una prueba o grupo de pruebas podrá suspenderse temporalmente cuando se presente alguna de las siguientes condiciones:

- El ambiente requerido no se encuentra disponible.
- La base de datos no está migrada o no contiene los datos mínimos.
- Las variables de entorno no están configuradas correctamente.
- El frontend o backend no inicia correctamente.
- Freightier Wallet no está configurado para Stellar/Soroban Testnet en la prueba manual de reventa.
- Una dependencia externa de prueba, como Stellar/Soroban Testnet, no responde adecuadamente.
- Se identifica un defecto bloqueante que impide continuar con el flujo evaluado.

La ejecución podrá reanudarse cuando la condición que generó la suspensión sea corregida, documentada o reemplazada por un escenario equivalente dentro del alcance del prototipo.

8. Casos de prueba

Esta sección presenta los casos de prueba definidos para validar los componentes y flujos principales de SecureTicket. Los casos se organizan por tipo de prueba y se relacionan con los elementos críticos del sistema: contratos inteligentes, backend/API, frontend, escáner QR, reventa en Testnet, sincronización on-chain/off-chain, seguridad, carga, despliegue y aceptación de usuario.

Los casos descritos en esta sección corresponden a pruebas implementadas, pruebas automatizadas listas para ejecución o guías manuales definidas para flujos que no se automatizan por restricciones técnicas, como la interacción con Freightier Wallet. El estado final de cada caso será actualizado después de la ejecución formal de las pruebas.

8.1. Formato de los casos de prueba

Para mantener consistencia en el registro de pruebas, cada caso se describe mediante los siguientes campos:

Campo	Descripción
ID	Identificador único del caso de prueba.
Nombre	Nombre corto del caso de prueba.
Tipo	Clasificación de la prueba: contrato, API, integración, E2E, carga, manual guiada, aceptación o documental.
Precondiciones	Condiciones necesarias antes de ejecutar la prueba.
Pasos resumidos	Secuencia general de acciones requeridas para ejecutar el caso.
Resultado esperado	Comportamiento que debe presentar el sistema para considerar la prueba aprobada.
Estado	Estado de ejecución de la prueba: pendiente, aprobada, fallida, parcial, bloqueada o no aplica.

Tabla 18: Formato utilizado para documentar casos de prueba

8.2. Casos de prueba de contratos inteligentes

Los casos de prueba de contratos inteligentes validan la lógica on-chain implementada en Soroban. Estas pruebas se ejecutan principalmente mediante `cargo test` y permiten verificar que las reglas críticas del ciclo de vida del ticket funcionen correctamente.

ID	Caso	Precondiciones	Resultado esperado	Estado
CONTRACT- ISSUE-01	Emisión de boleto	Contrato inicializado y datos válidos de evento/ticket.	El contrato crea un boleto único asociado al evento y al propietario correspondiente.	Pendiente
CONTRACT- RESALE-01	Reventa de boleto	Existe un boleto activo y propietario autorizado.	El contrato permite ejecutar la reventa bajo las reglas definidas y actualiza la propiedad del ticket.	Pendiente
CONTRACT- BURN-REMINT- 01	Versionamiento de ticket	Existe un ticket que será transferido en reventa.	La versión anterior queda no operable y se refleja una nueva versión o estado equivalente del ticket.	Pendiente
CONTRACT- COMMISSION-01	Distribución de comisiones	Existe una operación de reventa con precio y reglas de comisión definidas.	El contrato calcula y distribuye las comisiones según la lógica establecida.	Pendiente
CONTRACT- RESALE- CANCEL-01	Cancelación de reventa	Existe un ticket listado para reventa por su propietario.	El propietario puede cancelar la venta y el ticket deja de estar disponible en reventa.	Pendiente
CONTRACT- INVALIDATE-01	Invalidación de boleto	Existe un ticket activo o listado.	El boleto invalidado no puede venderse, listarse, comprarse ni redimirse posteriormente.	Pendiente
CONTRACT- REDEEM-01	Redención on-chain	Existe un ticket válido y un verificador autorizado.	El contrato permite la redención por verificador autorizado y rechaza redenciones no autorizadas o repetidas.	Pendiente
CONTRACT- NEG-01	Casos negativos de contrato	Existen entradas inválidas: ticket inexistente, usuario no autorizado, boleto usado o invalidado.	El contrato rechaza las operaciones inválidas sin alterar indebidamente el estado.	Pendiente

Tabla 19: Casos de prueba de contratos inteligentes

8.3. Casos de prueba backend/API

Los casos de prueba backend/API verifican los endpoints, reglas de negocio, seguridad funcional y validaciones realizadas del lado del servidor. Estas pruebas se ejecutan mediante el framework de pruebas del backend y Supertest o herramienta equivalente.

ID	Caso	Precondiciones	Resultado esperado	Estado
API-AUTH-01	Acceso sin token	Endpoint protegido disponible.	El sistema rechaza la solicitud con código 401 o equivalente.	Pendiente
API-AUTH-02	Token inválido o vencido	Se utiliza un token inválido, mal formado o expirado.	El sistema rechaza la solicitud y no ejecuta la operación.	Pendiente
SEC-ROLE-01	Validación de roles	Usuarios con roles <i>CUSTOMER</i> , <i>STAFF</i> y <i>ADMIN</i> disponibles.	Cada rol solo puede ejecutar las operaciones permitidas.	Pendiente
API-TICKET-OWNER-01	Operación sobre ticket ajeno	Usuario autenticado intenta operar un ticket que no le pertenece.	El backend rechaza la operación.	Pendiente
API-REQUEST-MANIP-01	Manipulación de solicitud	Solicitud con precio, estado o identificador manipulado.	El backend valida la información y rechaza operaciones inválidas.	Pendiente
API-SCAN-NEG-01	Escáner QR con casos negativos	Existen códigos QR válidos, inválidos, usados, expirados e inexistentes.	El escáner acepta códigos QR válidos y rechaza casos inválidos o no autorizados.	Pendiente
API-TICKET-INVALIDATE-01	Invalidación de ticket	Existe ticket activo o invalidable según la implementación.	El sistema bloquea operaciones posteriores sobre tickets inválidos o invalidados.	Parcial
API-RESALE-CANCEL-01	Cancelación de reventa	Existe ticket listado para reventa.	La cancelación retira el ticket del marketplace y evita compras posteriores.	Pendiente
API-CHECKIN-01	Registro de verificación	Ticket válido y usuario <i>STAFF/ADMIN</i> .	El sistema registra la verificación y evita duplicación en escaneos posteriores.	Pendiente

Tabla 20: Casos de prueba backend/API

8.4. Casos de prueba de integración

Los casos de integración validan la interacción entre backend, base de datos, indexador y eventos asociados a la lógica on-chain. En esta categoría se aceptan eventos normalizados o fixtures cuando la prueba no depende directamente de una transacción real en Testnet.

ID	Caso	Precondiciones	Resultado esperado	Estado
INT-SYNC-01	Sincronización de eventos	Base de datos disponible y eventos normalizados definidos.	Los eventos procesados actualizan correctamente PostgreSQL.	Pendiente
INT-SYNC-02	Evento de boleto creado	Evento de creación disponible.	Se crea o actualiza el registro del ticket con los campos esperados.	Pendiente
INT-SYNC-03	Evento de boleto listado	Evento de listado disponible.	El ticket queda marcado como disponible para reventa.	Pendiente
INT-SYNC-04	Evento de venta cancelada	Evento de cancelación disponible.	El ticket deja de estar listado para reventa.	Pendiente
INT-SYNC-05	Evento de boleto revendido	Evento de reventa disponible.	Se actualiza propietario, versión y estado del ticket.	Pendiente
INT-SYNC-06	Evento de boleto redimido	Evento de redención disponible.	El ticket queda marcado como usado o redimido según el modelo de datos.	Pendiente
INT-SYNC-07	Evento de boleto invalidado	Evento de invalidación disponible.	El ticket queda bloqueado para operaciones posteriores.	Pendiente
INT-SYNC-08	Evento duplicado	Se procesa dos veces el mismo evento.	El sistema mantiene idempotencia y no duplica registros.	Pendiente
INT-SYNC-09	Evento desconocido o mal formado	Evento no reconocido o con estructura inválida.	El sistema maneja el evento de forma controlada sin romper el proceso.	Pendiente

Tabla 21: Casos de prueba de integración

8.5. Casos de prueba frontend y end-to-end

Los casos frontend/E2E validan que los usuarios puedan completar flujos representativos desde la interfaz web del prototipo.

ID	Caso	Precondiciones	Resultado esperado	Estado
E2E-BUY-01	Compra primaria simulada	Usuario <i>CUSTOMER</i> , evento disponible y backend activo.	El usuario completa el checkout simulado y visualiza el ticket en inventario.	Pendiente
E2E-BUY-02	Usuario no autenticado intenta comprar	Usuario sin sesión activa.	El sistema bloquea la compra o redirige al inicio de sesión.	Pendiente
E2E-BUY-03	Error de checkout simulado	Condición de error controlada en checkout.	El sistema muestra un error comprensible y no deja estado inconsistente.	Pendiente
E2E-INVENTORY-01	Consulta de inventario	Usuario autenticado con tickets asociados.	El inventario muestra los tickets correspondientes al usuario.	Pendiente
E2E-SCAN-01	Escáner QR desde interfaz	Usuario <i>STAFF/ADMIN</i> y código QR válido disponible.	El escáner muestra éxito para código QR válido y registra el uso del ticket.	Pendiente
E2E-SCAN-02	Doble escaneo desde interfaz	Ticket previamente escaneado.	El sistema rechaza el segundo escaneo y muestra mensaje de error.	Pendiente
E2E-SCAN-03	Código QR inválido desde interfaz	Código QR inválido o manipulado.	El sistema rechaza el código QR y muestra un mensaje de error comprensible.	Pendiente
E2E-SCAN-04	<i>CUSTOMER</i> intenta acceder al escáner	Usuario con rol <i>CUSTOMER</i> .	El sistema bloquea el acceso a la funcionalidad de escáner.	Pendiente

Tabla 22: Casos de prueba frontend y end-to-end

8.6. Caso de prueba manual de reventa en Testnet

La reventa con Freightier se valida mediante una guía manual debido a la fragilidad de automatizar extensiones de wallet. Este caso representa el flujo central de la propuesta de SecureTicket.

Campo	Descripción
ID	E2E-REV-01
Nombre	Reventa manual guiada en Stellar/Soroban Testnet con Freightier.
Tipo	Manual guiada / Integración blockchain.
Precondiciones	Usuario A y Usuario B con cuentas y wallets de Testnet configuradas; Usuario A posee un ticket válido; contratos y backend apuntan a Stellar/Soroban Testnet; Freightier está instalado y configurado; PostgreSQL contiene los datos mínimos.
Pasos resumidos	Usuario A inicia sesión, conecta Freightier, lista un ticket propio para reventa; Usuario B inicia sesión, conecta Freightier y compra el ticket listado; se valida la firma, transacción en Testnet, cambio de propietario, actualización de inventario y consistencia con PostgreSQL.
Casos negativos	Compra del propio ticket, listado de ticket ajeno, compra de ticket cancelado, rechazo de firma en Freightier y wallet firmante distinta al usuario autenticado cuando aplique.
Resultado esperado	La reventa se completa en Stellar/Soroban Testnet, el ticket cambia de propietario, el marketplace e inventario se actualizan y los casos negativos son rechazados.
Estado	Pendiente de ejecución formal.

Tabla 23: Caso de prueba manual de reventa en Testnet

8.7. Casos de prueba de carga

Las pruebas de carga se organizan en tres escenarios: lectura pública, usuarios autenticados y operaciones críticas controladas. Estas pruebas se ejecutan sobre endpoints REST y no sobre mainnet, Freightier ni transacciones Soroban masivas.

ID	Escenario	Precondiciones	Resultado esperado	Estado
LOAD-PUBLIC-01	Lectura pública	Backend disponible, eventos y marketplace configurados.	Los endpoints públicos responden bajo carga moderada con tasa de error y p95 dentro del umbral.	Pendiente
LOAD-AUTH-01	Usuarios autenticados	Usuarios de prueba y credenciales válidas disponibles.	Login, inventario y endpoints autenticados responden dentro de los umbrales definidos.	Pendiente
LOAD-CRITICAL-01	Operaciones críticas controladas	Usuario <i>STAFF/ADMIN</i> , código QR o datos controlados y backend estable.	Escáner, checkout simulado u operaciones seleccionadas responden sin errores significativos.	Pendiente

Tabla 24: Casos de prueba de carga

8.8. Casos de prueba de smoke test de despliegue

El smoke test valida que el prototipo funcione de forma básica en el ambiente desplegado.

ID	Caso	Precondiciones	Resultado esperado	Estado
SMOKE-DEPLOY-01	Frontend disponible	URL de frontend desplegada.	La aplicación carga correctamente en navegador.	Pendiente
SMOKE-DEPLOY-02	Backend disponible	URL de backend desplegada.	El endpoint de salud responde correctamente.	Pendiente
SMOKE-DEPLOY-03	Login en despliegue	Usuario de prueba disponible.	El usuario puede iniciar sesión.	Pendiente
SMOKE-DEPLOY-04	Consulta de eventos y marketplace	Datos públicos disponibles.	Los eventos y marketplace cargan correctamente.	Pendiente
SMOKE-DEPLOY-05	Inventario y escáner	Usuario con tickets y usuario <i>STAFF/ADMIN</i> disponibles.	El inventario y el escáner funcionan según el rol correspondiente.	Pendiente

Tabla 25: Casos de prueba de smoke test de despliegue

8.9. Casos de prueba de aceptación de usuario

Las pruebas de aceptación se orientan a validar que usuarios o stakeholders puedan completar tareas representativas del prototipo y comprender los estados principales del ticket.

ID	Perfil	Tarea	Resultado esperado	Estado
UAT-01	Comprador	Iniciar sesión, consultar eventos, comprar ticket y revisar inventario.	El usuario completa la compra simulada y comprende el estado del ticket.	Pendiente
UAT-02	Vendedor/revendedor	Seleccionar ticket propio y listarlo para reventa.	El usuario comprende el proceso de publicación del ticket en marketplace.	Pendiente
UAT-03	Comprador de reventa	Consultar marketplace y comprar un ticket de reventa si el flujo está disponible.	El usuario comprende el flujo de reventa y el cambio de propiedad.	Pendiente
UAT-04	Staff/verificador	Escanear código QR válido e intentar doble escaneo.	El usuario comprende el resultado del escáner y la prevención de doble uso.	Pendiente
UAT-05	Admin/organizador	Revisar eventos, tickets o estados administrativos disponibles.	El usuario identifica la información operativa relevante del sistema.	Pendiente

Tabla 26: Casos de prueba de aceptación de usuario

8.10. Casos documentales de trazabilidad

La trazabilidad relaciona pruebas con requerimientos, casos de uso y atributos de calidad.

ID	Caso	Precondiciones	Resultado esperado	Estado
TRACE-01	Matriz de trazabilidad	Requerimientos, casos de uso y pruebas identificadas.	Cada prueba crítica se asocia con al menos un requerimiento, caso de uso o atributo de calidad.	Pendiente
TRACE-02	Cobertura parcial	Funcionalidades con cobertura parcial identificadas.	Las limitaciones quedan documentadas sin afirmar capacidades fuera del alcance.	Pendiente
TRACE-03	Exclusiones por alcance	Elementos fuera del alcance definidos.	Mainnet, pagos reales, carga blockchain masiva y automatización completa de Freightier quedan excluidos explícitamente.	Pendiente

Tabla 27: Casos documentales de trazabilidad

8.11. Resumen de cobertura de casos

La siguiente tabla resume la cobertura esperada por categoría de prueba.

Categoría	Casos principales	Cobertura esperada
Contratos inteligentes	CONTRACT-ISSUE, CONTRACT-RESALE, CONTRACT-REDEEM, CONTRACT-INVALIDATE	Lógica on-chain de emisión, reventa, redención, invalidación y casos negativos.
Backend/API	API-SCAN, SEC-ROLE, API-RESALE-CANCEL, API-TICKET-INVALIDATE	Reglas de negocio, seguridad funcional, escáner, cancelación e invalidación parcial.
Integración	INT-SYNC	Sincronización entre eventos normalizados y PostgreSQL.
Frontend/E2E	E2E-BUY, E2E-SCAN	Compra primaria simulada, inventario y escáner desde interfaz.
Reventa Testnet	E2E-REV-01	Flujo manual guiado de reventa con Freightier sobre Stellar/Soroban Testnet.
Carga	LOAD-PUBLIC, LOAD-AUTH, LOAD-CRITICAL	Rendimiento básico sobre endpoints REST representativos.
Despliegue	SMOKE-DEPLOY	Disponibilidad mínima de frontend, backend y flujos básicos.
Aceptación	UAT	Validación académica con usuarios o stakeholders.
Trazabilidad	TRACE	Relación entre pruebas, requerimientos, casos de uso y limitaciones.

Tabla 28: Resumen de cobertura esperada por categoría de prueba

9. Resultados y análisis

Esta sección presenta los resultados actualizados del proceso de pruebas de SecureTicket. La versión inicial del plan registraba varios bloqueos asociados principalmente a la base de datos, datos semilla, pruebas API, pruebas de integración, pruebas end-to-end reales, pruebas de carga autenticada, evidencia persistente y trazabilidad.

Después de ejecutar el backlog de cierre de calidad, los principales bloqueos automatizables fueron resueltos mediante la creación de un ambiente local aislado de pruebas con PostgreSQL en Docker, la aplicación de migraciones Prisma, la ejecución de suites automatizadas y la persistencia de evidencias en el repositorio.

Los resultados presentados en esta sección corresponden al estado actualizado del plan de pruebas. Se mantiene una distinción explícita entre pruebas automatizadas ejecutadas localmente, pruebas manuales pendientes y actividades que dependen de ambientes externos.

9.1. Resumen general de resultados

Después de la primera ejecución del plan de pruebas, algunas categorías habían quedado bloqueadas o parcialmente ejecutadas por problemas de infraestructura, datos de prueba o dependencia de ambientes externos. En particular, las pruebas Backend/API, integración/indexador y end-to-end real presentaban bloqueos asociados a la base de datos y al proceso de carga de datos semilla.

Para cerrar estos bloqueos, se realizó una segunda ejecución del backlog de calidad en un ambiente local aislado, utilizando PostgreSQL en Docker sobre la base de datos `stellar_tickets_test`. Este ambiente permitió ejecutar las pruebas automatizables sin depender de Supabase ni de datos del ambiente demo.

Como resultado, se cerraron los principales bloqueos automatizables del plan de pruebas. Las pruebas Backend/API pasaron de estar bloqueadas a ejecutarse exitosamente con 40 de 40 casos aprobados. Las pruebas de integración/indexador fueron ejecutadas dentro de la suite de API con 5 escenarios aprobados. La prueba end-to-end real, previamente bloqueada por datos semilla, fue ejecutada exitosamente con 1 de 1 caso aprobado. Adicionalmente, las pruebas unitarias generales del proyecto finalizaron con 100 de 100 casos aprobados.

También se ampliaron las pruebas de carga mediante dos escenarios ejecutados con k6:

carga autenticada con usuario *CUSTOMER* y operaciones críticas controladas sobre carrito/checkout vacío y escáner autenticado con código QR inválido. Ambos escenarios finalizaron con 0.00 % de errores y tiempos de respuesta p95 inferiores a 10 ms.

La Tabla 29 resume el estado actualizado de las categorías principales de prueba.

Categoría	Estado anterior	Estado actualizado
Base de datos de pruebas	No disponible / bloqueante	Aprobada localmente. PostgreSQL Docker disponible y migraciones Prisma aplicadas.
Backend/API	Bloqueado por base de datos	Aprobado localmente. Suite <code>npm run test:api</code> : 40/40 pruebas exitosas.
Integración/indexador	Bloqueado por base de datos	Aprobado localmente. Incluido en <code>test:api</code> : 5/5 pruebas exitosas.
End-to-end real	Bloqueado por error de datos semilla	Aprobado localmente. Suite <code>npm run e2e:real</code> : 1/1 prueba exitosa.
Escáner QR API	Parcial / bloqueado	Aprobado localmente dentro de la suite API.
Pruebas unitarias	Pendiente de consolidación final	Aprobadas localmente. Suite <code>npm run test:unit</code> : 100/100 pruebas exitosas.
Carga autenticada	No ejecutada	Aprobada localmente con k6: 8 VUs, 1 minuto, 1441 solicitudes, 0.00 % error, p95 global 8.54 ms.
Carga sobre operación crítica controlada	No ejecutada	Aprobada localmente con k6: 5 VUs, 1 minuto, 932 solicitudes, 0.00 % error, p95 global 9.06 ms.
Evidencia persistente	Parcial	Aprobada localmente. Se creó estructura <code>evidences/</code> y se persistieron logs backend, E2E y k6.
Matriz de trazabilidad	No versionada / no consolidada	Actualizada en <code>docs/test-traceability-matrix.md</code> . Requiere sincronización final con el documento de entrega.
Reventa Freightier/Testnet	No ejecutada	Pendiente manual. Requiere dos wallets, ejecución en Testnet y hash real de transacción.
Aceptación de usuario	No ejecutada	Pendiente manual. Requiere aplicación a usuarios representativos.
Smoke autenticado en staging	No ejecutado	Pendiente por ambiente externo. Requiere URL staging y usuarios <i>CUSTOMER</i> , <i>STAFF</i> y <i>ADMIN</i> .

Tabla 29: Resumen general actualizado de resultados de pruebas

9.2. Resultados de pruebas de contratos inteligentes

Las pruebas asociadas a la lógica de contratos inteligentes se consideran parte de la validación técnica del núcleo blockchain del prototipo. Estas pruebas verifican que las reglas principales del ciclo de vida del ticket se comporten de acuerdo con el alcance académico definido para SecureTicket.

El objetivo de esta categoría fue validar que las operaciones críticas asociadas a emisión, propiedad, transferencia, reventa, cancelación, invalidación y redención del ticket se comporten de forma consistente a nivel de lógica contractual. Esta validación es especialmente

relevante porque la propuesta del sistema se apoya en el uso de contratos inteligentes sobre Stellar/Soroban para garantizar trazabilidad y control de propiedad del ticket.

Elemento evaluado	Resultado	Observación
Emisión de tickets	Aprobado	Se validó la creación de tickets bajo las reglas definidas para el contrato.
Propiedad del ticket	Aprobado	Se verificó la asociación del ticket con su propietario correspondiente.
Reventa a nivel de contrato	Aprobado	Se validó la lógica contractual asociada al cambio de propiedad y reglas de reventa.
Cancelación de listado	Aprobado	Se verificó el comportamiento esperado al cancelar una publicación de reventa.
Invalidación/redención a nivel contractual	Aprobado	Se validó la lógica del contrato para estados finales del ticket.
Casos negativos contractuales	Aprobado	Se probaron restricciones y operaciones no permitidas de acuerdo con la lógica contractual.

Tabla 30: Resultados de pruebas de contratos inteligentes

La validación de contratos inteligentes permite afirmar que la lógica blockchain del prototipo se comporta de acuerdo con las reglas definidas para el alcance académico. Sin embargo, esta categoría no reemplaza la prueba manual de reventa con Freightier en Stellar/Soroban Testnet, ya que dicha prueba valida el flujo operativo completo con wallet, firma de transacción, hash real y actualización visible para el usuario.

Por tanto, la lógica contractual se considera validada a nivel de pruebas técnicas, mientras que la reventa completa con Freightier/Testnet permanece como pendiente manual.

9.3. Resultados de pruebas Backend/API

Las pruebas Backend/API fueron reejecutadas en un ambiente local aislado con PostgreSQL en Docker, utilizando la base de datos `stellar_tickets_test`. Este ambiente fue definido para evitar dependencia de Supabase o de datos del ambiente demo, y permitió cerrar el bloqueo de infraestructura identificado en la primera ejecución del plan de pruebas.

La configuración del ambiente incluyó el archivo `docker-compose.test.yml`, las variables de entorno de prueba en `backend/.env.test.example` y la ejecución de migraciones Prisma sobre la base de datos local. Una vez estabilizado el ambiente, se ejecutó la suite `npm run test:api`, obteniendo 40 pruebas exitosas de 40 ejecutadas.

La Tabla 31 presenta el resultado actualizado.

Elemento evaluado	Resultado	Observación
Ambiente de base de datos	Aprobado	PostgreSQL local en Docker disponible en 127.0.0.1:15432.
Migraciones Prisma	Aprobado	11 migraciones aplicadas sobre <code>stellar_tickets_test</code> .
Suite Backend/API	Aprobado	<code>npm run test:api</code> : 40/40 pruebas exitosas.
Autenticación y autorización	Aprobado	Validado dentro de la suite API.
Reglas de negocio backend	Aprobado	Validaciones ejecutadas sobre ambiente local aislado.
Escáner QR API	Aprobado	Cubierto dentro de la suite API.
Manejo de errores esperados	Aprobado	Validado mediante casos negativos incluidos en la suite.

Tabla 31: Resultados actualizados de pruebas Backend/API

Con esta ejecución, la categoría Backend/API deja de clasificarse como bloqueada. El bloqueo anterior por conectividad con PostgreSQL/Supabase fue resuelto mediante un ambiente local reproducible y aislado.

9.4. Resultados de integración e indexador

Las pruebas de integración e indexador fueron ejecutadas como parte de la suite `npm run test:api`. En la primera ejecución del plan, esta categoría había quedado bloqueada por problemas de conectividad con la base de datos. Para cerrar esta brecha, se configuró PostgreSQL local en Docker y se reejecutaron los casos implementados sobre una base de datos de pruebas aislada.

La suite actual validó 5 escenarios de integración/indexador y todos finalizaron exitosamente. Estos escenarios verifican el procesamiento y sincronización de información relevante entre la lógica backend, la persistencia en PostgreSQL y los eventos o estados asociados al ciclo de vida de los tickets.

Elemento evaluado	Resultado	Observación
Ejecución de suite de integración	Aprobado	Integrada dentro de <code>npm run test:api</code> .
Escenarios ejecutados	Aprobado	5/5 pruebas exitosas.
Procesamiento de eventos/estados	Aprobado	Validado sobre PostgreSQL local de pruebas.
Persistencia en PostgreSQL	Aprobado	Validada en <code>stellar_tickets_test</code> .
Bloqueo previo por base de datos	Resuelto	El bloqueo no se reprodujo en el ambiente local aislado.

Tabla 32: Resultados actualizados de integración e indexador

La categoría de integración/indexador pasa de estado bloqueado a estado aprobado local. La suite automatizada actual cubre los 5 escenarios implementados. Cualquier escenario

adicional no presente en la suite queda como candidato para ampliación futura, pero no bloquea la validación automatizada actual.

9.5. Resultados de frontend y pruebas end-to-end

La prueba end-to-end real había quedado bloqueada en la primera ejecución debido a un problema en los datos semilla de la base de datos. Específicamente, el bloqueo se encontraba asociado al proceso de carga de usuarios mediante `users.upsert` y a la falta de una condición de datos adecuada para ejecutar el flujo real.

Después de estabilizar el ambiente local de pruebas con PostgreSQL Docker y corregir la preparación de datos, se ejecutó la suite `npm run e2e:real`. La ejecución finalizó exitosamente con 1 de 1 prueba aprobada.

Elemento evaluado	Resultado	Observación
Preparación de datos E2E	Aprobado	El bloqueo de datos semilla fue corregido en el ambiente local.
Ejecución E2E real	Aprobado	<code>npm run e2e:real</code> : 1/1 prueba exitosa.
Trazas E2E	Aprobado	Evidencia persistida en <code>evidences/e2e/qa-p0-real-flow.log</code> .
Bloqueo previo	Resuelto	La prueba ya no queda bloqueada por base de datos ni por datos semilla.

Tabla 33: Resultados actualizados de pruebas frontend/E2E

Con esta ejecución, la categoría end-to-end real deja de considerarse bloqueada. El flujo fue validado en ambiente local controlado y la evidencia fue persistida en la carpeta de evidencias del repositorio.

9.6. Resultados de reventa en Stellar/Soroban Testnet

La prueba manual de reventa con Freightier en Stellar/Soroban Testnet permanece como una actividad pendiente de ejecución manual. Esta prueba no puede cerrarse únicamente con automatización local, debido a que requiere interacción con wallets reales de Testnet, firma de transacciones, ejecución sobre la red Stellar/Soroban y registro de evidencia observable del cambio de propietario.

La validación técnica de la lógica de reventa fue cubierta a nivel de contratos inteligentes y pruebas automatizadas locales. Sin embargo, el flujo operativo completo con Freightier debe ejecutarse manualmente para obtener evidencia final del comportamiento end-to-end en Testnet.

Elemento evaluado	Resultado	Observación
Guía manual de reventa	Pendiente	Debe ejecutarse con dos wallets Freightier en Testnet.
Firma de transacción con wallet	Pendiente	Requiere interacción manual del usuario con Freightier.
Hash de transacción	Pendiente	Debe registrarse como evidencia de ejecución on-chain.
Cambio de propietario	Pendiente	Debe validarse antes y después de la transacción.
Actualización de inventario	Pendiente	Debe verificarse en la interfaz o en la base de datos.
Actualización en PostgreSQL	Pendiente	Debe contrastarse con el estado off-chain del ticket.
Casos negativos de reventa	Pendiente	Deben probarse según disponibilidad del flujo manual.

Tabla 34: Estado de la prueba manual de reventa en Testnet

La evidencia mínima requerida para cerrar esta categoría incluye: wallet del vendedor, wallet del comprador, identificador del evento o ticket, contrato utilizado, hash de transacción, propietario antes y después, estado del inventario antes y después, estado del marketplace y registro correspondiente en PostgreSQL.

Esta prueba representa uno de los pendientes más relevantes del plan, porque la reventa con Freightier en Testnet corresponde al flujo central de la propuesta. Por tanto, mientras esta prueba no sea ejecutada, no debe afirmarse que el flujo completo de reventa fue validado integralmente en Testnet.

9.7. Resultados del escáner QR

El escáner QR fue validado a nivel API dentro de la suite `npm run test:api`, ejecutada contra PostgreSQL local de pruebas. En la primera ejecución, esta categoría había quedado parcialmente cubierta debido a que los casos API dependían de una base de datos funcional. Con la estabilización del ambiente local, el bloqueo fue cerrado y las pruebas asociadas al escáner quedaron cubiertas dentro de la suite API.

Elemento evaluado	Resultado	Observación
Escáner QR API	Aprobado	Cubierto dentro de <code>npm run test:api</code> .
Validaciones de autenticación	Aprobado	Incluidas en la suite API.
Validaciones de rol	Aprobado	Incluidas en la suite API.
Manejo de código QR inválido	Aprobado	Cubierto en pruebas API y en carga crítica controlada.
Evidencia persistente	Aprobado	Logs persistidos en <code>evidences/backend/qa-p0-api-indexer.log</code> .

Tabla 35: Resultados actualizados del escáner QR

Se mantiene como limitación de diseño que el flujo operativo del escáner QR es DB-first.

Esto significa que la verificación operativa del escáner se realiza contra el estado persistido en PostgreSQL, mientras que la redención on-chain se valida a nivel de contratos inteligentes y no como parte directa del flujo operativo del escáner. Esta decisión no corresponde a un bloqueo de pruebas, sino a una delimitación explícita del alcance implementado.

9.8. Resultados de pruebas de carga

Las pruebas de carga fueron ampliadas mediante dos escenarios ejecutados localmente con k6. La primera ejecución corresponde a carga autenticada con usuario *CUSTOMER*. La segunda corresponde a una operación crítica controlada compuesta por carrito/checkout vacío y escáner autenticado con código QR inválido.

Ambas pruebas se ejecutaron sobre el ambiente local del prototipo y finalizaron con 0.00 % de errores. Los tiempos de respuesta p95 globales se mantuvieron por debajo de 10 ms en ambos escenarios.

Escenario	VUs	Duración	Solicitudes	Tasa error	p95 global
Carga autenticada <i>CUSTOMER</i>	8	1 min	1441	0.00 %	8.54 ms
Operación crítica controlada	5	1 min	932	0.00 %	9.06 ms

Tabla 36: Resultados actualizados de pruebas de carga

Las evidencias de ejecución quedaron persistidas en las siguientes rutas relativas del repositorio:

- `evidences/k6/qa-p1-01-authenticated.log`
- `evidences/k6/qa-p1-01-authenticated.summary.json`
- `evidences/k6/qa-p1-02-critical-ops.log`
- `evidences/k6/qa-p1-02-critical-ops.summary.json`

El escenario del escáner fue ajustado a carga moderada debido a que el backend limita el endpoint `/api/admin/scan` a 60 solicitudes por minuto. Por esta razón, la prueba no busca demostrar carga masiva sobre el escáner, sino estabilidad bajo una carga controlada compatible con las restricciones del backend.

Con esta ejecución, la categoría de carga deja de clasificarse como no ejecutada. Se considera aprobada localmente para los escenarios definidos dentro del alcance académico del prototipo.

9.9. Resultados del smoke test de despliegue

El smoke test autenticado sobre staging real permanece como una actividad pendiente por dependencia de ambiente externo. Esta prueba requiere una URL de staging disponible y usuarios reales por rol: *CUSTOMER*, *STAFF* y *ADMIN*.

Durante el cierre del backlog automatizable se validó el comportamiento local del backend, E2E y carga moderada. Sin embargo, el smoke test autenticado en staging no fue ejecutado porque depende de credenciales y ambiente externo de despliegue.

Elemento evaluado	Resultado	Observación
URL staging real	Pendiente externo	Requiere disponibilidad del ambiente de despliegue.
Usuario <i>CUSTOMER</i> en staging	Pendiente externo	Requiere credencial real o cuenta semilla en staging.
Usuario <i>STAFF</i> en staging	Pendiente externo	Requiere credencial real o cuenta semilla en staging.
Usuario <i>ADMIN</i> en staging	Pendiente externo	Requiere credencial real o cuenta semilla en staging.
Login autenticado en staging	Pendiente externo	No ejecutado por falta de ambiente o credenciales.
Inventario en staging	Pendiente externo	No ejecutado por falta de ambiente o credenciales.
Escáner en staging	Pendiente externo	No ejecutado por falta de ambiente o credenciales.
Bloqueo de <i>CUSTOMER</i> en escáner	Pendiente externo	Debe validarse cuando exista staging autenticado.

Tabla 37: Estado del smoke test autenticado en staging

Este pendiente no invalida la validación local automatizada del prototipo, pero limita la afirmación de despliegue autenticado en un ambiente externo. Por tanto, el documento debe indicar que el smoke test autenticado en staging queda pendiente por disponibilidad de ambiente y credenciales reales.

9.10. Resultados de aceptación de usuario

Las pruebas de aceptación de usuario permanecen pendientes de ejecución. Esta categoría no puede cerrarse únicamente mediante automatización, porque requiere usuarios representativos que interactúen con el prototipo y diligencien un formulario de evaluación.

Debido a que el proyecto no cuenta con un stakeholder comercial formal o un cliente

externo disponible para validar el sistema, la estrategia recomendada consiste en realizar una aceptación académica con usuarios representativos. Estos participantes no actuarán como clientes reales ni como representantes comerciales de una plataforma de boletería, sino como usuarios externos al equipo de desarrollo que simulan los roles principales del sistema.

Elemento evaluado	Resultado	Observación
Participantes	Pendiente	Se recomienda aplicar aceptación de usuario a 5 o 6 usuarios representativos.
Rol comprador	Pendiente	Debe validar exploración de evento, compra simulada e inventario.
Rol vendedor/revendedor	Pendiente	Debe validar publicación de ticket en reventa.
Rol staff/verificador	Pendiente	Debe validar escaneo o verificación de código QR.
Rol administrador/organizador	Pendiente	Debe validar consulta operativa de eventos, tickets o estados.
Formularios recibidos	Pendiente	Deben consolidarse los formularios diligenciados.
Tasa de tareas completadas	Pendiente	Debe calcularse después de aplicar la prueba.
Comentarios abiertos	Pendiente	Deben consolidarse observaciones y mejoras sugeridas.

Tabla 38: Estado actualizado de pruebas de aceptación de usuario

El criterio mínimo recomendado para cerrar esta categoría es contar con al menos 5 participantes, cubrir los roles principales del sistema y obtener una tasa de tareas completadas exitosamente o parcialmente igual o superior al 80%. También se recomienda calcular promedios de claridad de interfaz, facilidad de uso y comprensión del estado del ticket.

Mientras esta prueba no sea aplicada, no debe afirmarse aceptación de usuario. La formulación correcta del estado actual es: aceptación de usuario pendiente de ejecución con usuarios representativos en ambiente académico controlado.

9.11. Resultados de trazabilidad

La matriz de trazabilidad fue actualizada y versionada en el repositorio mediante el archivo `docs/test-traceability-matrix.md`. Esta matriz relaciona los casos de prueba ejecutados con las categorías funcionales y técnicas evaluadas durante el cierre del backlog de calidad.

En la primera ejecución, la trazabilidad se encontraba pendiente de consolidación formal. Después de la actualización del backlog, se registraron los resultados reales de las pruebas automatizadas locales, incluyendo Backend/API, integración/indexador, E2E real, escáner QR API, pruebas unitarias, pruebas de carga y evidencias persistidas.

Elemento	Estado	Observación
Matriz de trazabilidad	Actualizada	Archivo disponible en <code>docs/test-traceability-matrix.md</code> .
Relación prueba-evidencia	Parcialmente consolidada	Se asociaron resultados reales del backlog automatizable.
Backend/API	Trazado	Suite <code>test:api</code> : 40/40.
Integración/indexador	Trazado	5/5 escenarios dentro de <code>test:api</code> .
E2E real	Trazado	Suite <code>e2e:real</code> : 1/1.
Pruebas unitarias	Trazado	Suite <code>test:unit</code> : 100/100.
Carga autenticada y crítica	Trazado	Evidencias k6 persistidas en <code>evidences/k6</code> .
Reventa Freightier/Testnet	Pendiente	Requiere ejecución manual y hash de transacción.
Aceptación de usuario	Pendiente	Requiere aplicación a usuarios representativos.
Smoke staging autenticado	Pendiente externo	Requiere URL y credenciales reales de staging.

Tabla 39: Resultados actualizados de trazabilidad

La trazabilidad se considera actualizada para las pruebas automatizadas ya ejecutadas. Se mantiene pendiente su cierre total hasta incorporar las evidencias manuales de reventa Freightier/Testnet, aceptación de usuario y, si se dispone del ambiente, smoke autenticado en staging.

9.12. Fallos, bloqueos y limitaciones identificadas

Después de la segunda ejecución del backlog de calidad, varios bloqueos reportados inicialmente fueron resueltos mediante la creación de un ambiente local aislado con PostgreSQL Docker, la reejecución de pruebas automatizadas y la persistencia de evidencias.

Las Tablas 40 y 41 presentan el estado actualizado de los bloqueos y limitaciones identificadas.

Elemento	Estado anterior	Estado actualizado
Base de datos de pruebas	No disponible / bloqueante	Resuelto. PostgreSQL Docker disponible y migraciones Prisma aplicadas.
Backend/API	Bloqueado por PostgreSQL/Supabase	Resuelto. <code>npm run test:api</code> : 40/40 pruebas exitosas.
Integración/indexador	Bloqueado por base de datos	Resuelto. 5/5 pruebas aprobadas dentro de <code>test:api</code> .
End-to-end real	Bloqueado por error de datos semilla	Resuelto. <code>npm run e2e:real</code> : 1/1 prueba exitosa.
Escáner QR API	Parcial / bloqueado	Resuelto a nivel API local dentro de <code>test:api</code> .
Pruebas unitarias	Pendiente de consolidación final	Resuelto. <code>npm run test:unit</code> : 100/100 pruebas exitosas.
Carga autenticada	No ejecutada	Resuelta localmente con k6: 1441 solicitudes, 0.00 % error, p95 8.54 ms.
Carga sobre operación crítica	No ejecutada	Resuelta localmente con k6: 932 solicitudes, 0.00 % error, p95 9.06 ms.
Evidencia persistente	Parcial	Resuelta localmente. Se creó evidences/ con logs backend, E2E y k6.
Matriz de trazabilidad	No versionada	Actualizada en <code>docs/test-traceability-matrix.md</code> .

Tabla 40: Bloqueos automatizables resueltos durante la segunda ejecución

Elemento	Estado anterior	Estado actualizado
Reventa Freightier/Testnet	No ejecutada	Pendiente manual. Requiere dos wallets Testnet, ejecución real y hash de transacción.
Aceptación de usuario	No ejecutada	Pendiente manual. Requiere aplicación a usuarios representativos.
Smoke autenticado en staging	No ejecutado	Pendiente por ambiente externo. Requiere URL staging y usuarios reales por rol.
Escáner QR on-chain directo	Parcial por diseño	Se mantiene como limitación de alcance: escáner operativo DB-first y redención on-chain validada a nivel de contrato.
Mainnet, pagos reales y KYC/AML	Fuera de alcance	Se mantienen fuera del alcance académico del prototipo.

Tabla 41: Pendientes, dependencias externas y limitaciones de alcance restantes

Con esta actualización, los bloqueos automatizables principales fueron cerrados. Las limitaciones restantes corresponden a actividades manuales, dependencias de ambiente externo o delimitaciones explícitas del alcance académico del prototipo.

9.13. Análisis general

La segunda ejecución del backlog de calidad permitió cerrar los principales bloqueos automatizables identificados durante la primera fase del plan de pruebas. La causa principal

de los bloqueos anteriores era la dependencia de una base de datos remota o no estabilizada para pruebas. Para resolver este problema, se creó un ambiente local aislado con PostgreSQL en Docker, definido en `docker-compose.test.yml`, junto con variables de entorno de prueba documentadas en `backend/.env.test.example`.

Este ambiente permitió ejecutar las pruebas sin contaminar Supabase, el ambiente demo o datos de despliegue. Sobre esta base se ejecutaron exitosamente las pruebas Backend/API, integración/indexador, end-to-end real, escáner QR API, pruebas unitarias y pruebas de carga moderada.

Los resultados obtenidos muestran una mejora sustancial frente a la primera ejecución del plan. La suite Backend/API pasó de estar bloqueada a aprobar 40 de 40 pruebas. Las pruebas de integración/indexador fueron ejecutadas con 5 de 5 casos exitosos. El E2E real, previamente bloqueado por datos semilla, aprobó 1 de 1 caso. Las pruebas unitarias generales aprobaron 100 de 100 casos. Finalmente, las pruebas de carga autenticada y de operación crítica controlada finalizaron con 0.00 % de error y tiempos p95 globales inferiores a 10 ms.

La evidencia fue persistida en la carpeta `evidences/`, organizada por tipo de prueba. Además, se actualizaron documentos operativos del repositorio, incluyendo `docs/operations/QA_P1_RESULTS`, `docs/operations/LOAD_TESTING.md`, `docs/operations/QA_BACKLOG_EXECUTION.md` y `docs/test-trac`.

A pesar de este avance, todavía existen actividades que no pueden cerrarse únicamente mediante automatización local. La prueba de reventa con Freightier en Stellar/Soroban Testnet requiere ejecución manual con dos wallets reales de Testnet y registro de hash de transacción. Las pruebas de aceptación de usuario requieren participantes representativos y formularios diligenciados. El smoke test autenticado en staging depende de la disponibilidad de URL y credenciales reales para usuarios *CUSTOMER*, *STAFF* y *ADMIN*.

En consecuencia, el estado actualizado del plan permite afirmar que la validación automatizada local del prototipo fue cerrada para las categorías principales de backend, integración, E2E, escáner, unitarias y carga moderada. No obstante, la validación integral completa del flujo de reventa requiere aún la ejecución manual con Freightier/Testnet y la aplicación de pruebas de aceptación de usuario con participantes representativos.

9.14. Conclusión de resultados

Con base en la segunda ejecución del backlog de calidad, SecureTicket cuenta con una validación automatizada local sólida para los componentes principales del prototipo. Los

bloqueos iniciales relacionados con base de datos, integración, pruebas API y E2E real fueron resueltos mediante la creación de un ambiente local aislado y reproducible.

Los resultados más relevantes son los siguientes:

- PostgreSQL local de pruebas fue configurado correctamente con Docker y migraciones Prisma.
- La suite Backend/API aprobó 40 de 40 pruebas.
- La integración/indexador aprobó 5 de 5 pruebas dentro de la suite API.
- La prueba end-to-end real aprobó 1 de 1 caso.
- Las pruebas unitarias aprobaron 100 de 100 casos.
- Las pruebas de carga autenticada y de operación crítica controlada finalizaron con 0.00 % de error.
- La evidencia fue persistida en la carpeta `evidences/`.
- La matriz de trazabilidad fue actualizada en `docs/test-traceability-matrix.md`.

Por tanto, las categorías previamente bloqueadas por infraestructura dejan de considerarse brechas abiertas. El plan de pruebas pasa a reflejar una validación automatizada local cerrada para el alcance técnico principal del prototipo.

Se mantienen pendientes tres elementos no automatizables localmente: la reventa manual con Freightier en Stellar/Soroban Testnet, las pruebas de aceptación con usuarios representativos y el smoke test autenticado en staging real. Estos pendientes no invalidan los resultados automatizados obtenidos, pero sí delimitan el alcance de la afirmación de validación integral.

La conclusión actual es que SecureTicket fue validado satisfactoriamente en ambiente local controlado para backend, integración, end-to-end, escáner, pruebas unitarias y carga moderada. La validación final del flujo central de reventa debe completarse con evidencia manual en Testnet.

10. Riesgos y mitigaciones

Esta sección presenta los principales riesgos identificados durante la planeación, implementación y ejecución de pruebas de SecureTicket, junto con las acciones de mitigación

propuestas. Los riesgos se clasifican según su impacto sobre la ejecución del plan de pruebas, la validez de los resultados y la capacidad de demostrar los flujos críticos del prototipo.

Dado que SecureTicket depende de componentes locales, servicios desplegados, base de datos remota, contratos inteligentes, Stellar/Soroban Testnet y Freighter Wallet, algunos riesgos están asociados a infraestructura externa o condiciones de ambiente que pueden afectar la ejecución formal de las pruebas sin necesariamente representar defectos funcionales del sistema.

10.1. Riesgos identificados

Las Tablas 42, 43 y 44 presentan los riesgos identificados durante el ciclo de pruebas.

ID	Riesgo	Prob.	Impacto	Descripción
R-01	Indisponibilidad de base de datos remota	Alta	Alto	Las pruebas de API, integración y E2E real pueden bloquearse si PostgreSQL/Supabase no es accesible desde el ambiente de pruebas.
R-02	Datos de prueba incompletos o inconsistentes	Media	Alto	La ausencia de usuarios, tickets, roles, códigos QR o restricciones de base de datos puede impedir la ejecución de flujos E2E o generar fallos en datos semilla.
R-03	Reventa Testnet no ejecutada	Media	Alto	La no ejecución de la prueba manual de reventa con Freighter limita la validación end-to-end del flujo central del proyecto.
R-04	Dependencia de Freighter Wallet	Media	Medio	La interacción con Freighter puede ser difícil de automatizar y depende de configuración manual de cuentas, red y fondos de Testnet.
R-05	Inestabilidad o latencia de Stellar/Soroban Testnet	Media	Medio	Las pruebas blockchain pueden verse afectadas por disponibilidad, latencia o cambios en servicios de Testnet.

Tabla 42: Riesgos identificados durante el ciclo de pruebas, parte 1

ID	Riesgo	Prob.	Impacto	Descripción
R-06	Escáner operativo DB-first	Alta	Medio	El escáner no ejecuta redención on-chain en el flujo operativo, lo que puede generar confusión si no se documenta adecuadamente.
R-07	Invalidación administrativa incompleta	Media	Medio	No existe un endpoint administrativo completo de invalidación, por lo cual la cobertura queda distribuida entre contrato, reglas backend y validaciones de flujos.
R-08	Pruebas de carga incompletas	Media	Medio	Si solo se ejecuta un smoke corto, no se puede afirmar que el sistema fue validado bajo carga moderada representativa.
R-09	Aceptación de usuario no ejecutada	Media	Medio	Sin usuarios o stakeholders participantes, no es posible extraer conclusiones de aceptación del prototipo.
R-10	Matriz de trazabilidad no actualizada	Media	Medio	Sin trazabilidad, resulta difícil demostrar qué requerimientos, casos de uso o atributos de calidad fueron cubiertos por cada prueba.

Tabla 43: Riesgos identificados durante el ciclo de pruebas, parte 2

ID	Riesgo	Prob.	Impacto	Descripción
R-11	Evidencia no persistida	Media	Medio	Si los logs, reportes, capturas o trazas no se guardan, los resultados no quedan suficientemente soportados para el documento final.
R-12	Diferencia entre ambiente local y staging	Media	Alto	Un flujo puede aprobar localmente pero fallar en staging por variables de entorno, migraciones, datos o restricciones de infraestructura.
R-13	Uso accidental de configuración fuera del alcance	Baja	Alto	Configurar mainnet, activos reales o endpoints productivos violaría el alcance académico definido para el proyecto.
R-14	Cambios de código después de ejecutar pruebas	Media	Alto	Cambios posteriores a la ejecución pueden invalidar resultados si no se congela un commit o rama específica.
R-15	Fallos no funcionales no detectados	Media	Medio	La ausencia de pruebas profundas de seguridad, accesibilidad o escalabilidad puede dejar riesgos fuera del alcance de las pruebas actuales.

Tabla 44: Riesgos identificados durante el ciclo de pruebas, parte 3

10.2. Plan de mitigación

Las Tablas 45, 46 y 47 presentan las acciones propuestas para reducir el impacto de los riesgos identificados.

ID	Riesgo asociado	Mitigación propuesta	Prioridad
R-01	Indisponibilidad de base de datos remota	Preparar una base de datos de prueba accesible, validar conexión antes de ejecutar pruebas API e integración, y documentar variables de entorno requeridas.	Alta
R-02	Datos de prueba incompletos o inconsistentes	Crear o actualizar datos semilla con usuarios, roles, eventos, tickets, códigos QR y wallets de Testnet. Validar migraciones y restricciones únicas antes del E2E real.	Alta
R-03	Reventa Testnet no ejecutada	Ejecutar la guía manual E2E-REV-01 con Freightier, Usuario A, Usuario B, ticket válido, contrato de Testnet y verificación de propietario antes/después.	Alta
R-04	Dependencia de Freightier Wallet	Mantener la reventa como prueba manual guiada, con condiciones claras, checklist, cuentas de Testnet y pasos reproducibles.	Media
R-05	Inestabilidad o latencia de Testnet	Ejecutar pruebas blockchain en ventanas estables, registrar hash de transacción cuando aplique y separar resultados de contrato local de resultados sobre Testnet.	Media

Tabla 45: Plan de mitigación de riesgos, parte 1

ID	Riesgo asociado	Mitigación propuesta	Prioridad
R-06	Escáner operativo DB-first	Documentar explícitamente que el escáner funciona DB-first y que la redención on-chain se valida a nivel de contrato mediante CONTRACT-REDEEM-01.	Alta
R-07	Invalidación administrativa incompleta	Clasificar la cobertura como parcial justificada. Validar invalidación en contrato y reglas backend; no crear endpoint fuera del diseño solo para pruebas.	Media
R-08	Pruebas de carga incompletas	Ejecutar los tres escenarios k6 definidos: lectura pública, usuarios autenticados y operaciones críticas controladas, con credenciales y datos seguros.	Media
R-09	Aceptación de usuario no ejecutada	Aplicar la plantilla de aceptación a un grupo mínimo de usuarios o stakeholders, registrar tareas, resultados, observaciones y promedios.	Media
R-10	Matriz de trazabilidad no actualizada	Versionar la matriz de trazabilidad y asociar cada prueba crítica con requerimientos, casos de uso o atributos de calidad del SRS/SAD.	Alta

Tabla 46: Plan de mitigación de riesgos, parte 2

ID	Riesgo asociado	Mitigación propuesta	Prioridad
R-11	Evidencia no persistida	Guardar logs, reportes k6, trazas Playwright, capturas, hashes de transacción y formularios en una carpeta de evidencia con fecha, commit y ambiente.	Alta
R-12	Diferencia local/staging	Ejecutar smoke test de despliegue antes de pruebas E2E reales y validar que variables, migraciones y datos sean consistentes entre ambientes.	Alta
R-13	Configuración fuera del alcance	Verificar explícitamente que todas las pruebas blockchain usen Stellar/Soroban Testnet y que no se utilicen activos reales, mainnet ni pagos productivos.	Alta
R-14	Cambios posteriores al código	Congelar rama, commit o tag antes de la ejecución formal de pruebas y registrar ese identificador en el documento de resultados.	Alta
R-15	Fallos no funcionales no detectados	Declarar explícitamente que pentest, certificación productiva, mainnet, pagos reales y auditorías externas quedan fuera del alcance del plan.	Media

Tabla 47: Plan de mitigación de riesgos, parte 3

10.3. Riesgos materializados durante la ejecución

Durante la ejecución de pruebas se materializaron algunos riesgos previamente identificados. Estos riesgos no necesariamente representan defectos funcionales del sistema, pero sí limitaron la cobertura alcanzada en la fase de ejecución.

ID	Riesgo materializado	Efecto observado	Acción recomendada
RM-01	Base de datos remota no accesible	Las pruebas API e integración quedaron bloqueadas por imposibilidad de conexión a PostgreSQL/Supabase.	Reconfigurar ambiente de prueba con una base de datos accesible y repetir pruebas API e integración.
RM-02	Restricción de base de datos faltante en datos semilla	La prueba E2E real quedó bloqueada por error en <code>users.upsert</code> , asociado a una restricción única faltante.	Ajustar migración o datos semilla para garantizar unicidad y repetir el E2E real.
RM-03	Reventa Freightier no ejecutada	No se generó hash de transacción ni validación de cambio de propietario en Testnet.	Ejecutar manualmente E2E-REV-01 con usuarios, wallets y contrato de Testnet.
RM-04	Carga autenticada no ejecutada	Solo se ejecutó un smoke corto de lectura pública con k6.	Configurar credenciales/tokens y ejecutar escenarios autenticados y operaciones críticas controladas.

Tabla 48: Riesgos materializados durante la ejecución, parte 1

ID	Riesgo materializado	Efecto observado	Acción recomendada
RM-05	Aceptación de usuario no ejecutada	No se obtuvieron resultados de aceptación de usuario.	Aplicar formulario de aceptación y consolidar resultados por rol y tarea.
RM-06	Matriz de trazabilidad no versionada	No se pudo evaluar formalmente cobertura contra requerimientos o casos de uso.	Reconstruir, versionar y actualizar matriz de trazabilidad.
RM-07	Evidencia general no persistida	Algunos resultados solo quedaron como salida de consola, sin archivo persistente.	Reejecutar comandos redirigiendo salidas a archivos de evidencia.

Tabla 49: Riesgos materializados durante la ejecución, parte 2

10.4. Plan de contingencia

En caso de que alguno de los riesgos vuelva a presentarse durante una nueva ejecución, se aplicarán las siguientes medidas de contingencia:

- **Falla de base de datos remota:** ejecutar las pruebas contra una base de datos local o una instancia temporal de PostgreSQL con migraciones y datos semilla controlados.
- **Falla de datos semilla:** corregir las restricciones y datos iniciales antes de repetir pruebas E2E reales o de integración.
- **Falla de Testnet o Freightier:** mantener la validación de contratos mediante **cargo test** y reprogramar la prueba manual de reventa en una ventana estable.
- **Falta de credenciales para carga o smoke test:** preparar usuarios de prueba específicos para lectura autenticada, escáner y operaciones críticas controladas.
- **Falta de participantes para aceptación de usuario:** ejecutar la validación con compañeros o stakeholders académicos que no hayan implementado directamente el flujo evaluado.
- **Ausencia de evidencia persistente:** repetir la ejecución de los comandos con redirección a archivos de log y almacenar los resultados en una carpeta identificada por fecha y commit.
- **Diferencias entre documentación e implementación:** clasificar la funcionalidad como parcial o fuera de alcance, evitando afirmar comportamientos no soportados por el prototipo.

10.5. Riesgo residual

Después de aplicar las mitigaciones propuestas, algunos riesgos permanecen como residuales debido al alcance académico del prototipo. Estos riesgos no se eliminan completamente, pero se documentan para delimitar correctamente la interpretación de los resultados.

Riesgo residual	Descripción	Tratamiento
No validación en mainnet	El sistema no se prueba en Stellar Mainnet ni con activos reales.	Se declara fuera del alcance.
Escáner sin redención on-chain operativa	El escáner valida uso del ticket en modalidad DB-first.	Se documenta como decisión de arquitectura del prototipo.
Reventa con Freighter manual	La prueba depende de ejecución manual por la naturaleza de la wallet.	Se acepta como prueba manual guiada.
Sin pentest formal	No se ejecuta una auditoría de seguridad externa.	Se limita a pruebas de seguridad funcional.
Carga limitada a API REST	No se ejecuta carga masiva sobre transacciones blockchain.	Se justifica por alcance y representatividad.
Aceptación académica	La aceptación se realiza en contexto académico, no comercial.	Se declara como validación controlada de prototipo.

Tabla 50: Riesgos residuales

10.6. Conclusión de riesgos

Los riesgos más relevantes para el cierre del proceso de pruebas se relacionan con la disponibilidad de base de datos, la ejecución de la reventa manual en Testnet, la preparación de datos de prueba, la persistencia de evidencia y la actualización de trazabilidad. La mayoría de estos riesgos pueden mitigarse mediante una nueva ejecución controlada, congelando el commit evaluado, preparando una base de datos estable y almacenando evidencia de forma sistemática.

Las limitaciones asociadas a mainnet, pagos reales, redención on-chain desde el escáner y carga blockchain masiva no se consideran defectos del sistema, sino delimitaciones explícitas del alcance académico de SecureTicket.

11. Matriz de trazabilidad

La matriz de trazabilidad permite relacionar los casos de prueba definidos en este documento con los requerimientos, casos de uso, atributos de calidad o elementos funcionales

que buscan validar. Su propósito es evidenciar que las pruebas no se encuentran aisladas, sino que responden directamente a los objetivos del sistema y al alcance definido para SecureTicket.

Debido a la extensión de la matriz completa y a la cantidad de columnas requeridas para registrar identificador, nombre, tipo, nivel, requerimiento asociado, estado de implementación, estado de ejecución y observaciones, dicha matriz no se incluye directamente en el cuerpo principal del documento. Para mantener la legibilidad del plan de pruebas, la matriz completa se presenta como anexo en la sección **Anexo F: Matriz completa de trazabilidad**. En esta sección se presenta únicamente la definición de criterios, los estados utilizados, el resumen de cobertura y el análisis de brechas.

Dado que la matriz completa no se encontraba versionada al momento de la primera ejecución formal de pruebas, el anexo correspondiente se plantea como una matriz base reconstruida a partir de los casos de prueba definidos en el plan. El estado de ejecución deberá actualizarse cuando se realice una nueva ejecución formal con ambiente estable, evidencia persistente y resultados consolidados.

11.1. Criterios de trazabilidad

Para este plan de pruebas, cada caso se relaciona con al menos uno de los siguientes elementos:

- **Requerimiento funcional:** funcionalidad esperada del sistema, como compra, re-venta, escáner, inventario o autenticación.
- **Requerimiento no funcional:** atributo de calidad asociado a seguridad, rendimiento, confiabilidad, trazabilidad o integridad.
- **Caso de uso:** interacción principal entre un actor y el sistema.
- **Elemento arquitectónico:** componente relevante de la solución, como contrato inteligente, backend, frontend, base de datos o indexador.
- **Limitación de alcance:** comportamiento que se valida parcialmente o que se excluye explícitamente del prototipo.

11.2. Estados utilizados en la matriz

La matriz utiliza dos tipos de estado: estado de implementación y estado de ejecución. Estos estados permiten diferenciar si una prueba existe o está definida, y si además fue ejecutada formalmente durante la fase de validación.

Estado	Descripción
Implementada	La prueba existe en el repositorio o como guía manual versionada.
Lista para ejecución	La prueba está definida y preparada, pero no ha sido ejecutada formalmente.
Ejecutada	La prueba fue ejecutada y cuenta con resultado registrado.
Bloqueada	La prueba no pudo ejecutarse por problemas de ambiente, datos o dependencias externas.
Parcial	La prueba cubre solo una parte del flujo o una parte del requerimiento debido al alcance real del prototipo.
No aplica	La prueba o funcionalidad queda fuera del alcance académico del proyecto.
No versionada	La prueba, guía o matriz no se encuentra actualmente en el repositorio, aunque haya sido definida previamente.

Tabla 51: Estados utilizados en la matriz de trazabilidad

11.3. Ubicación de la matriz completa

La matriz completa de trazabilidad se presenta en el **Anexo F: Matriz completa de trazabilidad**, debido a que su tamaño excede el formato recomendado para el cuerpo principal del documento. Esta decisión permite mantener el capítulo de trazabilidad como una sección analítica y evitar la inclusión de una tabla extensa que interrumpa la lectura del informe.

La matriz anexada debe conservar, como mínimo, las siguientes columnas:

- ID de prueba.
- Nombre del caso de prueba.
- Tipo de prueba.
- Nivel de prueba.
- Requerimiento, caso de uso o atributo de calidad asociado.
- Estado de implementación.
- Estado de ejecución.

- Observaciones.

La matriz completa debe actualizarse cuando se realice una nueva ejecución formal de pruebas, especialmente para los casos actualmente bloqueados, parciales o no ejecutados.

11.4. Resumen de cobertura por categoría

La siguiente tabla resume la cobertura general identificada a partir de los casos de prueba definidos y de los resultados obtenidos durante la ejecución.

Categoría	Estado de cobertura	Análisis
Contratos inteligentes	Fuerte	Las pruebas de contrato fueron ejecutadas y aprobaron, cubriendo emisión, reventa, redención, cancelación, invalidación y casos negativos.
Backend unitario	Fuerte	Las pruebas unitarias del backend aprobaron y validan reglas internas del sistema.
Backend/API	Implementada pero bloqueada	Las pruebas API están implementadas, pero no fueron ejecutadas por indisponibilidad de PostgreSQL/Supabase.
Integración/indexador	Implementada pero bloqueada	La sincronización con eventos normalizados está definida, pero no pudo ejecutarse por bloqueo de base de datos.
Frontend/E2E estándar	Aceptable	Los flujos UI estándar aprobaron, incluyendo compra simulada, inventario y escáner básico.
E2E real	Bloqueada	El flujo real quedó bloqueado por un problema de datos semilla asociado a una restricción única faltante.
Reventa Testnet	No ejecutada	No se validó manualmente el flujo con Freightier ni se generó hash de transacción.
Escáner QR	Parcial	Cubierto en UI/unitarias y contrato para redención; API completa bloqueada por base de datos.
Carga	Parcial	Solo se ejecutó un smoke corto de lectura pública; los escenarios autenticado y crítico no se ejecutaron.
Smoke despliegue	Parcial	Se validó disponibilidad pública de frontend, backend y eventos; no se ejecutaron flujos autenticados.
Aceptación de usuario	No ejecutada	Existe plantilla, pero no fue aplicada a usuarios o stakeholders.
Trazabilidad	Reconstruida en anexo	La matriz no estaba versionada durante la ejecución; se reconstruye como base documental en el anexo correspondiente.

Tabla 52: Resumen de cobertura por categoría

11.5. Análisis de brechas de trazabilidad

A partir del resumen anterior y de la matriz completa ubicada en el anexo, se identifican las siguientes brechas principales:

- **Reventa end-to-end en Testnet:** aunque la lógica de reventa está cubierta en contratos, no se ejecutó el flujo manual con Freightier. Por tanto, falta validar el flujo completo con usuarios, wallet y Stellar/Soroban Testnet.
- **API e integración:** las pruebas están implementadas, pero se encuentran bloqueadas por conectividad con PostgreSQL/Supabase. Esto impide cerrar la trazabilidad de escáner API, registro de verificación, roles e indexador.
- **E2E real:** el flujo real quedó bloqueado por un problema de datos semilla relacionado con `users.upsert`. Esto limita la validación contra ambiente real.
- **Carga autenticada y crítica:** los scripts existen, pero no se ejecutaron por falta de tokens, credenciales y datos seguros.
- **Aceptación de usuario:** la aceptación se encuentra preparada, pero no aplicada. Por tanto, no se pueden extraer conclusiones de percepción o usabilidad con participantes.
- **Trazabilidad versionada:** la matriz no estaba versionada en el repositorio durante la ejecución; el anexo sirve como reconstrucción documental, pero debe versionarse para el cierre formal del plan de pruebas.

11.6. Conclusión de trazabilidad

La trazabilidad evidencia que SecureTicket cuenta con cobertura fuerte en contratos inteligentes, backend unitario y frontend E2E estándar. También muestra que existen pruebas implementadas para backend/API, integración, escáner, carga y aceptación de usuario, aunque algunas no fueron ejecutadas por bloqueos de ambiente o falta de datos.

La matriz completa se conserva como anexo debido a su extensión y a su formato horizontal. Esta decisión permite mantener el cuerpo principal del documento enfocado en el análisis de cobertura, mientras que el detalle completo de cada caso de prueba queda disponible para revisión en el anexo correspondiente.

Para cerrar completamente la trazabilidad del plan de pruebas, se recomienda ejecutar nuevamente las pruebas bloqueadas con una base de datos accesible, versionar la matriz en el repositorio, ejecutar la reventa manual con Freightier sobre Stellar/Soroban Testnet y aplicar la prueba de aceptación a usuarios o stakeholders.

12. Conclusiones

El proceso de pruebas de SecureTicket permitió estructurar y evaluar el estado de calidad del prototipo desde diferentes niveles: contratos inteligentes, backend, frontend, integración, escáner QR, carga, despliegue y aceptación de usuario. A partir de los resultados obtenidos, se evidencia que el sistema cuenta con una base técnica sólida en las pruebas de contratos Soroban, pruebas unitarias del backend y pruebas unitarias del frontend. En particular, las pruebas de contratos aprobaron en su totalidad y cubrieron funcionalidades críticas como emisión, reventa, cancelación, invalidación, redención, verificadores autorizados y casos negativos relevantes.

Las pruebas unitarias del backend y del frontend también presentaron resultados favorables, lo cual respalda la correcta implementación de componentes internos y comportamientos principales de la interfaz. Adicionalmente, las pruebas end-to-end estándar permitieron validar flujos representativos del prototipo, como compra primaria simulada, inventario y escáner QR desde la interfaz web.

Sin embargo, no todos los criterios de aceptación definidos en el plan pudieron cerrarse completamente durante la ejecución reportada. Las pruebas de API e integración quedaron bloqueadas por problemas de conectividad con PostgreSQL/Supabase, mientras que la prueba end-to-end real quedó bloqueada por un problema de datos semilla asociado a una restricción única faltante en base de datos. Estos bloqueos no representan fallas funcionales confirmadas del sistema, pero sí limitan la evidencia disponible para validar formalmente los endpoints críticos, el indexador, el escáner API y los flujos reales contra ambiente integrado.

También se identificaron pruebas relevantes no ejecutadas, principalmente la reventa manual en Stellar/Soroban Testnet con Freightier y las pruebas de aceptación de usuario. La no ejecución de la reventa en Testnet representa una brecha importante, debido a que este flujo corresponde al núcleo de la propuesta de SecureTicket. Aunque la lógica de reventa fue validada a nivel de contrato inteligente, todavía se requiere validar el flujo completo con usuarios, wallet, Testnet, cambio de propietario y actualización de inventario.

En términos de alcance, el sistema no debe interpretarse como una solución productiva ni como una implementación sobre mainnet. Las pruebas se ejecutaron dentro del marco académico definido para el proyecto, excluyendo mainnet, pagos reales, activos con valor económico, carga masiva sobre blockchain y operación comercial. De igual forma, el escáner QR se valida bajo un enfoque DB-first, mientras que la redención on-chain se

encuentra cubierta a nivel de contrato inteligente.

En conclusión, SecureTicket presenta una validación fuerte a nivel de contratos inteligentes y pruebas unitarias, además de una validación parcial pero útil en frontend E2E, escáner UI, smoke test y carga pública básica. Para afirmar una validación integral del prototipo, es necesario resolver los bloqueos de base de datos, ejecutar nuevamente las pruebas API e integración, realizar la prueba manual de reventa en Testnet con Freighter, aplicar pruebas de aceptación de usuario y versionar la matriz de trazabilidad. Con estas actividades pendientes, el proceso de pruebas quedaría suficientemente respaldado para cerrar la validación funcional, técnica y operativa del prototipo dentro del alcance académico definido.

12.1. Trabajo futuro

Como trabajo futuro del proceso de pruebas, se recomienda ejecutar una segunda iteración formal con una base de datos de prueba estable y accesible, de manera que puedan completarse las pruebas API, integración, indexador y end-to-end real que quedaron bloqueadas durante la ejecución inicial. Esta iteración debe incluir la persistencia sistemática de logs, reportes, trazas, capturas y resultados en una carpeta de evidencia asociada al commit evaluado.

También se recomienda ejecutar la prueba manual de reventa en Stellar/Soroban Testnet con Freighter, registrando el contrato utilizado, hash de transacción, propietario antes y después, estado del ticket, inventario y consistencia con PostgreSQL. Esta prueba es prioritaria porque permite cerrar el flujo central de reventa segura propuesto por el sistema.

En cuanto al escáner QR, se recomienda mantener documentada la decisión arquitectónica DB-first y, como mejora futura, evaluar la integración directa de la redención on-chain dentro del flujo operativo del escáner. Esta mejora no es obligatoria para el alcance actual del prototipo, pero permitiría fortalecer la trazabilidad blockchain del proceso de validación en puerta.

Para las pruebas de carga, se recomienda ejecutar los tres escenarios definidos: lectura pública, usuarios autenticados y operaciones críticas controladas. Esto permitiría pasar de un smoke corto de rendimiento a una validación más representativa del comportamiento del backend bajo carga moderada. No se recomienda ejecutar carga masiva sobre Freighter ni sobre transacciones Soroban reales, ya que esto no sería representativo del alcance académico del prototipo.

Finalmente, se recomienda aplicar las pruebas de aceptación de usuario a un grupo mínimo de participantes con roles representativos, como comprador, vendedor/revendedor, staff y administrador. Los resultados de esta validación deben consolidarse en la matriz de trazabilidad, junto con las pruebas técnicas, para relacionar los hallazgos con los requerimientos, casos de uso y atributos de calidad del sistema.

13. Anexos

Los anexos reúnen las evidencias, archivos de soporte y artefactos complementarios utilizados durante la ejecución y análisis de las pruebas de SecureTicket. Estos elementos permiten respaldar los resultados reportados en el documento y facilitar la revisión posterior de comandos, salidas, reportes, trazabilidad, formularios y decisiones de alcance.

13.1. Anexo A: Evidencias de pruebas de contratos inteligentes

Este anexo debe incluir la evidencia asociada a la ejecución de pruebas de contratos Soroban mediante `cargo test`. Se recomienda anexar:

- Salida de consola de `cargo test`.
- Resumen de pruebas ejecutadas, aprobadas y fallidas.
- Identificación del commit evaluado.
- Advertencias o *warnings* no bloqueantes.
- Relación de casos cubiertos: emisión, reventa, versionamiento, comisiones, cancelación, invalidación, redención, verificadores autorizados y casos negativos.

Estado del anexo: pendiente de adjuntar evidencia persistente. Durante la ejecución reportada se obtuvo salida de consola, pero no se generó archivo de log persistente.

13.2. Anexo B: Evidencias de pruebas backend/API e integración

Este anexo debe contener los soportes relacionados con pruebas unitarias del backend, pruebas API, pruebas del escáner QR, validaciones de roles, integración con base de

datos e indexador.

Se recomienda incluir:

- Salida de consola de `npm run test:unit`.
- Salida de consola de `npm run test:api`.
- Logs de errores de conexión a PostgreSQL/Supabase, si aplican.
- Evidencia de pruebas bloqueadas por infraestructura.
- Resultados de pruebas de reglas backend, roles, tokens, escáner, registros de verificación, cancelación e invalidación parcial.
- Evidencia de pruebas de integración del indexador cuando se ejecuten con base de datos accesible.

Estado del anexo: parcial. Las pruebas unitarias backend fueron ejecutadas exitosamente, pero las pruebas API e integración quedaron bloqueadas por conexión a base de datos.

13.3. Anexo C: Evidencias de pruebas frontend y end-to-end

Este anexo debe incluir las evidencias de pruebas unitarias frontend, pruebas end-to-end estándar y pruebas end-to-end reales.

Se recomienda anexar:

- Salida de consola de `npm test`.
- Salida de consola de `npm run e2e`.
- Salida de consola de `E2E_REAL=1 npm run e2e:real`.
- Trazas generadas por Playwright.
- Archivos `trace.zip` o `error-context.md`, cuando existan.
- Capturas o registros de flujos: compra primaria simulada, inventario, escáner QR, código QR inválido, doble escaneo y bloqueo de usuarios con rol *CUSTOMER*.

Estado del anexo: parcial. Las pruebas frontend unitarias y E2E estándar fueron ejecutadas, mientras que el E2E real quedó bloqueado por un problema de datos semilla en base de datos.

13.4. Anexo D: Evidencias de reventa en Stellar/Soroban Testnet

Este anexo debe contener los soportes de la prueba manual guiada de reventa con Freightier en Stellar/Soroban Testnet.

Se recomienda incluir:

- Guía manual E2E-REV-01.
- Usuarios de prueba utilizados, sin exponer credenciales.
- Wallets de Testnet utilizadas, sin exponer claves privadas.
- Dirección del contrato utilizado.
- Hash de transacción de la operación de reventa.
- Capturas del marketplace antes y después.
- Capturas del inventario del Usuario A y Usuario B.
- Estado del ticket antes y después en PostgreSQL.
- Resultado de casos negativos: compra del propio ticket, listado de ticket ajeno, compra de ticket cancelado y rechazo de firma.

Estado del anexo: pendiente. La prueba manual de reventa en Testnet no fue ejecutada durante la fase reportada.

13.5. Anexo E: Evidencias de pruebas de carga

Este anexo debe incluir los reportes generados por k6 para los escenarios de carga definidos.

Se recomienda anexar:

- Script `tests/load/public-read.k6.js`.
- Script de usuarios autenticados, si aplica.
- Script de operaciones críticas controladas, si aplica.
- Salida de consola de k6.
- Reporte JSON o resumen exportado.
- Tabla de métricas por escenario: usuarios virtuales, duración, solicitudes, tasa de error, promedio, p95 y máximo.
- Análisis de limitaciones: ausencia de carga sobre mainnet, Freightier y transacciones Soroban masivas.

Estado del anexo: parcial. Se ejecutó únicamente un smoke corto de lectura pública. Los escenarios autenticado y crítico quedaron pendientes.

13.6. Anexo F: Matriz completa de trazabilidad

Este anexo contiene la matriz completa de trazabilidad de pruebas de SecureTicket. La matriz relaciona cada caso de prueba con su tipo, nivel, requerimiento o caso de uso asociado, estado de implementación, estado de ejecución y observaciones.

Debido a su extensión y orientación horizontal, la matriz se presenta como anexo para no afectar la legibilidad del cuerpo principal del documento. La matriz completa debe conservar, como mínimo, las siguientes columnas:

- ID de prueba.
- Nombre del caso de prueba.
- Tipo de prueba.
- Nivel de prueba.
- Requerimiento, caso de uso o atributo de calidad asociado.
- Estado de implementación.
- Estado de ejecución.

- Observaciones.

Estado del anexo: pendiente de versionamiento. La matriz fue reconstruida documentalmente, pero no se encontraba versionada en el repositorio durante la ejecución reportada.

13.7. Anexo G: Evidencias de smoke test de despliegue

Este anexo debe contener las evidencias de disponibilidad del ambiente desplegado.

Se recomienda incluir:

- URL del frontend desplegado.
- URL del backend desplegado.
- Respuesta del endpoint `/health`.
- Evidencia de carga de eventos.
- Evidencia de carga del marketplace.
- Evidencia de login, inventario y escáner cuando se ejecuten con credenciales de prueba.
- Commit desplegado y fecha de validación.

Estado del anexo: parcial. Se validó disponibilidad pública de frontend, backend, eventos y marketplace, pero no se ejecutaron flujos autenticados en staging.

13.8. Anexo H: Formularios de aceptación de usuario

Este anexo debe incluir los instrumentos y resultados de las pruebas de aceptación de usuario.

Se recomienda anexar:

- Plantilla del formulario de aceptación.
- Tareas asignadas por rol: comprador, vendedor/revendedor, staff y administrador.
- Formularios diligenciados.
- Tabla de resultados consolidados.
- Promedios por pregunta.
- Observaciones abiertas de los participantes.

- Conclusión de aceptación del prototipo.

Estado del anexo: pendiente. La plantilla existe, pero no fue aplicada a usuarios o stakeholders durante la fase reportada.

13.9. Anexo I: Limitaciones y decisiones de alcance

Este anexo debe registrar las decisiones de alcance que delimitan la interpretación de las pruebas.

Se recomienda incluir:

- Exclusión de Stellar Mainnet.
- Exclusión de pagos reales.
- Exclusión de activos con valor económico.
- Exclusión de procesos KYC/AML.
- Exclusión de carga masiva sobre blockchain.
- Justificación del escáner DB-first.
- Justificación de redención on-chain validada a nivel de contrato.
- Justificación de reventa con Freightier como prueba manual guiada.
- Justificación de invalidación administrativa como cobertura parcial.

Estado del anexo: documentado conceptualmente en el plan de pruebas. Debe consolidarse como soporte final si el documento se entrega con anexos separados.